

Bachelor Thesis

Communication issues: Down the rabbit hole of language interoperability

by

Kai Erik Niermann

(2720905)

First supervisor: Atze Van der Ploeg
Daily supervisor: Atze Van der Ploeg
Second reader: Verano Merino

February 18, 2025

*Submitted in partial fulfillment of the requirements for
the VU degree of Bachelor of Science in Computer Science*

Communication issues: Down the rabbit hole of language interoperability

Kai Erik Niermann

Vrije Universiteit Amsterdam
Amsterdam, The Netherlands
k.e.niermann@student.vu.nl

ABSTRACT

Your Master Thesis has to be written in English and should follow the ACM style (see the [Overleaf template](#)). We also suggest a *structured abstract* following the structure below.

Context. Write this at the end

Goal. Write this at the end

Method. Write this at the end

Results. Write this at the end

Conclusions. Write this at the end

As a very rough indication, the final thesis report typically entails, excluding appendixes, between 10 and 30 pages, with an average of 15 pages. The large variation depends on many variables including the specific field, project nature, and context. We advise to ask your supervisor if you should consider which number as a reference.

1 INTRODUCTION

Abstractly speaking interoperability in the context of software is the simple notion of two or more systems, such as languages following different design paradigms in their execution models and linguistic properties, working together via some means of communication. Interoperability is by no means a new concept, in many ways, it ties quite naturally into how systems we interact with evolve. If we take the most relevant example of such systems, programming languages, we can note that their evolution over time has been based on the development of abstractions to make their usage easier for programmers. Naturally, with the evolution of these abstractions we developed systems that must translate composited abstractions which define most software back into the basic codes that machines can apply to execute said abstractions. Two notable byproducts of this type of evolution are that

- (1) The performance characteristics in kind evolved with the abstractions we developed to design new programming languages.
- (2) Not all abstractions are created equal, so naturally languages evolve in various directions, built on various execution models.

From the first point, it would then be a natural question, sooner or later, for most programmers how one can create a balance between ease of use abstractions and still the desired performance characteristics of the granular low-level control of hardware through languages that allow such control. For the second point a question one might ask is in what way do these abstractions differ, and in what ways are these differences more applicable to some domains than others. For both of the above questions, I hope that it becomes clear how we then derive the desire for interoperability in software systems.

In a more contemporary context, we have had a considerable evolution from the early days of interoperability. The increasing variations of hardware chipsets have led to a corresponding variety of instruction set architectures (ISAs). Similarly, when zooming out we have had a considerable increase in the number of programming languages, largely supported by improvements in language execution models. As languages and ISAs evolved compilers naturally adopted a modular or specialized architecture to better fit the various designs people wanted to implement. We saw the evolution of ahead-of-time (AOT) compilers used predominantly for statically typed languages with a greater focus on performance as opposed to language flexibility. Likewise here we saw interoperability applied in various forms, most notably with the creation of LLVM Intermediate Representation (IR) the use of a language agnostic IR meant optimization passed could be performed on arbitrary languages as long as developers implemented the corresponding IR generation. We also saw the creation of just-in-time (JIT) compilers, which themselves provided an abstraction to interpret languages and compile them to machine code at runtime. This generally comes at the cost of some performance by opening the door to a substantial field of research into different types of optimizations one could apply given information only accessible at runtime. A notable downside with effective JIT compilers has been their historically complex nature and inherent lack of modularity. JIT compilers generally exist alongside a language runtime, examples being the JVM for Java, V8 for JavaScript, and CLR for C#, to execute said languages with an emphasis on dynamic behavior through lacking the modularity in their ability to work with a wider variety of languages as contrasted with AOT compilers they generally have no analog to IR. Nonetheless there has been a decently sized body of research dedicated into exploring improvements in both the performance characteristics of JIT compilers to more closely match those of AOT compilers, but also at various attempts for create language agnostic representations not only to yield a more modular architecture to support the development of new languages on the respective language platform but also to allow for effect polyglot (multi-language) systems whose dynamics are all nicely understood and defined within the context of some language runtime.

The initial idea I wanted to test came from a curiosity about the interactions between languages operating under different execution models, particularly the dynamics of the Julia JIT compiler and the Clang AOT compiler used with C++ in a practical context. This interest gradually evolved into a broader inquiry about software interoperability, particularly exploring its practical aspects and why they are intriguing and warrant further investigation. My research followed a two-pronged approach: the first being to develop a comprehensive understanding of the modern state of software

interoperability and the diverse ways polyglot systems are implemented. Here, my goal was to present a broad structural overview, motivated by a desire to understand how my practical work fits within the larger field and how its techniques are compared to industry standards. The second prong involved acquiring hands-on knowledge by applying interoperability concepts to a specific case. I aimed to explore whether I could replace a performance-intensive segment of my program with one written in a different language, highlighting both the benefits and drawbacks of modern techniques compared to legacy methods. The crux was to bridge the gap between theoretical frameworks and practical applications, examining how abstract concepts contrast with real-world examples. Ultimately, this research sought to link and contrast various aspects of interoperability theory with an engaging practical case, providing insights into how these concepts connect and reinforce each other.

The primary aim of this work is to present a large structural overview of the modern state of software interop and to link and contrast various concepts from this with an applied practical example. The goal of this is to create a somewhat comprehensive overview of the world of language interoperability while providing a concrete example that applies some modern approaches. Similarly to various other fields in computer science interoperability nowadays forms a rather expansive collection of research and real-world/industry applications. While this is of course broadly speaking positive I believe there are insights to be gained from creating a more unified albeit constrained picture of the larger field. Generally speaking, it is not too uncommon that you can see a dichotomy form in different aspects of computer science between what works and has worked for a while; often taking the form of legacy systems which are iteratively patched to be good enough; and newer research which can often be somewhat inaccessible, complex to adapt, and difficult to see the real utility of. The same can indeed be seen for the case of software interoperability, thus with this work, I hope to gain but also provide some insights into how some of these newer developments in the design of programming languages, interoperability layers, and polyglot systems can potentially assist in creating more effective software solutions.

To put it more concretely, there are several objectives I aim to achieve with this work:

- (1) Research and present an overview of the modern state of language interoperability, with a focus on the more recent developments in the field and compare and contrast these findings with an applied example.
- (2) Examine how various aspects of language interoperability have evolved together, thus presenting a more unified and interconnected view of the field.
- (3) Assess the benefits and drawbacks of employing modern techniques, particularly focusing on language paradigms within interoperability, the performance dynamics in critical systems, and the influence of modern type systems on the design and application of interoperability frameworks.
- (4) Using insights gained from both the theoretical and practical aspects to yield potential future directions for research and development in the field of software interoperability.

One could theoretically go into an arbitrary depth when doing this type of exploration but due to the inherently flexible nature

that exists not only within computer science but any field that has developed for so long there is value in creating a more abstract overview which can hopefully provide a more stable foundation over time. The primary limitation is that naturally with something as expansive as interoperability while there is the aim to document a wide portion of the field, not all aspects will be discussed, the focus will be on the modern state of the field as this isn't meant to be a historical document, rather a review of the modern state of interoperability in software. The main purpose again is to provide the reader with both a broad overview of interoperability in software but also link this to a modern concrete example to present some potentially interesting new areas to explore especially in the context of evolving type systems of programming languages.

To form an outline of how this overview of interop. Will be approached it would be reasonable to build on the fundamental aspects of the field which act as the foundation for various legacy approaches in the implementation of polyglot (multi-language) systems. From this, we can expand to how higher levels of abstractions are applied in terms of both the evolution of programming languages specifically focusing on their type systems and the corresponding execution models. The methodology here will be focused on aggregating and reviewing various instances of research and applications to demonstrate this overview. A focus will be placed on specifically how various aspects of interoperability build on top of one another and tie into each other, with a secondary focus on ensuring that sufficient topics are introduced to provide the background knowledge required to understand the motivating practical example and give the reader the same insights into where it fits in the broader context of interoperability as I wished to gain with this research. Of note is that this is not meant to be a meta-analysis but rather a survey that is compared and contrasted against a concrete practical implementation.

The background and related works will first build upon some of the more low-level aspects of interop, mainly foreign function interfaces (FFI) and the corresponding machine code specifications known as application binary interfaces (ABIs). We will then shift to the more modern approaches employing various meta techniques largely revolving around the application of various types of abstractions to define different modes of interop. Finally, a motivating practical example will be presented which applies various of the topics introduced in the background to demonstrate their application in addition to contrasting this with various other approaches. The hope here is to provide not only a broad overview but also compare and contrast this with a practical example so the reader can gain a broad overview to explore some potentially interesting areas of interoperability, but also understands how various aspects of modern interoperability can have demonstrable real-world utility through an applied example and aren't only theoretical demonstrations.

2 BACKGROUND & RELATED WORK

2.1 FFI & Low Level Interop

2.1.1 Foreign Function Interface (FFI). FFI is a generic term which describes any implementation which enables a piece of code written in one language to call functions written in another. As the name implies these implementations usually provide some interface; that is, abstractions; which enable a programmer to make

calls into some binary compiled into some target language differing from the source. In practice FFI can be thought of as existing on two layers, the Application Binary Interface (ABI) level and the Application Programming Interface (API) level. The ABI level describes some common standard for how to compile source code into machine code. The API level then describes the corresponding high-level counterpart which comes in the form of libraries which expose means of working with this underlying shared ABI as to enable communication in a shared low level standard. By far the most common ABI which many languages can compile to and by extension have interoperability with is the C ABI. One slight tangent, naturally when we talk about an ABI we are referring to machine level concepts, which is to say that while its become the de-facto standard to say ‘C ABI’ the underlying standard’s we are talking about are the platform specific ABI standards which C compilers correspond to. It’s just more straightforward to say there is a universal ‘C ABI’ as practically speaking this is more or less accurate.

To jump straight into a little demonstration for how the programmer facing API version of something like this commonly works we can look at some examples.

```
1 // mylib.c
2 #include <stdio.h>
3
4 void greet(const char* name) {
5     printf("Hello, %s!\n", name);
6 }
```

Listing 1: FFI Example in C

```
1 ccall(:greet, "mylib"), Cvoid, (CString,), "World")
```

Listing 2: FFI Example in Julia

```
1 #[link(name = "mylib")]
2 extern "C" { fn greet(name: *const c_char); }
3
4 fn main() {
5     let name = "World";
6     let c_name = CString::new(name).expect("CString::new failed");
7     unsafe {
8         greet(c_name.as_ptr());
9     }
10 }
```

Listing 3: FFI Example in Rust

```
1 mylib = ctypes.CDLL('./libmylib.so')
2
3 mylib.greet.argtypes = [ctypes.c_char_p]
4 mylib.greet.restype = None
5
6 name = b"World"
7 mylib.greet(name)
```

Listing 4: FFI Example in Python

One critical thing to observe is in all instance we must ensure the FFI boundary method and its arguments conform to the memory layout of the target language as specified by the ABI. Since string

types for example generally have different underlying representations depending on the language here we must ensure the string has the form of a C-string, that is its a basic pointer to an array of characters *char**.

This in turn limits FFI boundary methods by the syntax and underlying ABI of our target language. For example, as C has no generics, we cannot utilize a generic method as an FFI boundary despite generics existing in our source language and Rust being compatible with the C ABI.

```
1 extern "C" {
2     fn add<T>(a: T, b: T) -> T;
3 }
```

Listing 5: Pseudocode example of extern C generics

Stated differently FFI boundary method signatures must mirror those of the target language. We can create some flexibility here by abstracting the boundary utilizing wrappers; which make defining the boundary closer to that of the source language while implicitly ensuring that the actual underlying boundary is valid, an example of this is the `@ccall` macro in Julia. But this doesn’t really address the underlying issue that our FFI boundary limits the idiomatic application of our source and target languages.

2.1.2 Application Binary Interface (ABI). ABI stability is a key factor to understanding the current landscape of not only interoperability between languages but also within different versions of languages themselves. As the name implies the stability of an ABI refers to its specification, that being how a compiler generates machine code, being stable or unchanging. An ABI being ‘fully stable’ implies that all aspects of its specifications will not change with any future modification of its execution system (JIT compiler, AOT compiler, etc.). One of the primary reasons for so many languages having interop with C is due to the fact that it has full ABI stability, thus there exist essentially a promise that a compiler can always generate the same machine code for a given source language that conforms to the defined C ABI standard knowing this will never some day break due to changes in C’s calling conventions, memory layout, etc.

The other side of this is ABI instability, most languages currently have what could be considered partial ABI stability. What this means in practice is that most languages have some execution system which generates machine code, commonly this version or a set of versions have a common ABI. Within this set of versions there exists stability, meaning shared library calls; which are reliant on ABI stability; can function with any other languages capable of producing machine code which conforms to this ABI. Though across different ABI’s this communication breaks down as we no longer have a homogenous standard generating the machine code but conflicting and often incompatible standards.

This nature of partial stability in the ABI’s of most languages creates one of the primary limitations to language interoperability with FFI, due to the rather complex and to an extent controversial nature of having an ABI which is both fully specified and consistent across versions there are not alot of languages which have this and by extension not alot of compilers which chose to implement an ABI for this. Furthermore there is the point that if for some set of

languages there is an existing C ABI already implemented, this in turn implies we have communication between these two languages with the common language taking the form of the C ABI. It should be clarified that while interop in many languages occurs with the C ABI as the basic common ground.

2.1.3 C-ABI and Interop in practice. To describe how interop utilizing the C ABI actually plays out in practice lets express two arbitrary languages as L_1 and L_2 . In the case of primitives the generated machine code usually produces matching layouts, thus little additional work is involved beyond defining the FFI boundaries in both languages¹. As start wanting to pass more complex things across a language boundary that might not have as straightforward of a representation in C we run into the first issue with pure C ABI based interop. This being that it leads to considerable manual specification and redefining language specific syntax, otherwise called marshalling or boxing through the FFI boundary and in turn unmarshalling in the target language. Additionally we must ensure that the representation of our type in L_1 matches that in L_2 which for a large library could mean having to manually specific all types such that we establish a mirrored relationship; while this is somewhat obvious that it has to happen the manual nature of it would be less then desirable in the case of very large libraries we wish to interop with.

To address these issues of having to manually box everything such that it is compatible with the C ABI various language features and libraries exist as abstractions to enable more seamless and simplified interop between languages. The common features with these interoperability abstraction systems are usually automatic binding generation through parsing and implicit boxing and unboxing of complex types that cant be trivially passed through the FFI boundary. Examples of this are Rusts CXX library [13] or the way Swift implemented interop as a direct part of its compiler.

Through these abstractions interop between many languages has become a mostly trivial task, in that most of the manual activities are now generally automated in one way or another, though as stated before due to the fact that this is all ultimately still working through the C-ABI (or some slight modification of it) we are still very much limited in the things we can actually pass between languages. Though these limitations are generally not large enough to really discourage the usage of FFI as the primary means of interop.

So to briefly summarize we can see that FFI generally relies on a common denominator ABI between languages; often the C ABI; and a set of tools to automate any manual boxing and unboxing such that the developer can; for the most part; simply define a boundary in both languages and use said boundary as if it were a regular call. An important final consideration with these FFI systems is how optimization works across language boundaries. Since the methods of shared libraries are only resolved at link time no optimizations except LTO can occur. The shared object file from the target language has optimizations applied from its own compiler then depending on if the compiler has Link Time Optimization (LTO) implemented we can optimize the whole program across the

language boundary, otherwise no optimizations occurs across the language boundary.

2.1.4 WebAssembly (WASM). WASM offers an interesting insights into how language interop is developing on the web. First announced in 2015 and then released in 2017 WASM is a byte-code format designed to be executed within web-browsers using stack-based Virtual Machines (VM) though has now also been implemented to run both via AOT and JIT compilers. The WASM specification was designed by the World Wide Web Consortium (W3C) but has various compiler implementations by different groups. WASM when used as a compilation target for other languages not only allows potentially arbitrary languages to run on the web, but through its ABI also allows for another common denominator of communication. To understand the state of its ABI we have to examine the different compiler toolchains that target WASM. Currently the major ones are

What we can understand from this is that it seems WASI will be the ABI to which wasi-wasm will be the likely target most compilers aim to ultimately achieve, furthermore WASI seems to aim for stability with its canonical ABI, which could suggest that we might see a new common standard of interoperability, especially as wasm compilers evolve. One distinguishing feature of specifically WASI's canonical ABI is development of the **component model**, the component model refers to type definitions written in *WASM Interface Type* (WIT), which is an interface description language (IDL) used to express interface; that is, types and functions; in a language agnostic manner[5]. From the defined WIT code we can generate the corresponding bindings in our language thus mitigating the need for any type-shenanigan's in the case of interop since our generated bindings will by default be compatible with the ABI. It should be noted that as the documentation specifies WIT implements contracts, that is similar to how an *extern* block expresses a function which must be ABI compatible so to do the generated bindings from the WIT code all inherently imply compatability with the underlying ABI.

Thus if you have say some library compiled to wasi-wasm which utilizes the component model to specify the API you can in turn use the library specification to generate ABI compliant bindings for your source code, then use a wasi-wasm compiler and have full cross-language interoperability without the need for any complex wrapper libraries. Though once again here we are limited by the expressiveness of the component model and corresponding ABI, meaning that if a library wants to make something available for use externally it has to conform to the component model thus potentially limiting certain language specific features for a cross language setting. Unless similarly to the C ABI we adjust the compiler to support these language specific features and in turn break compatibility with the ABI.

2.2 Binding generators

2.2.1 Binding generator designs. It should be clarified that in the previous instances of interop I for the most part avoided mentioning the exact details of how different type systems are mapped between languages. In the context of FFI based interop there are two key considerations in regards to types one must consider, that being the kind of type we are bridging and the parameter-passing strategy.

¹This is somewhat of an oversimplification but for the most part holds true as there is generally either a uniform expectation of how primitives are represented for most languages or this is trivially apparent from the language specification (i.e. `uint64_t`).

Compiler/Toolchain	Description
Emscripten	This was the initial project to define a POSIX-compliant ABI for WASM, although it has not achieved stability as of 2020.
Cheerp	Supports WASI-WASM as a target for the compiler. It inherently provides interoperability with JavaScript by compiling to JS and utilizes ts2cpp for generating C++ interfaces from TypeScript types.
WASI	A subgroup within the WASM community group aiming to define a set of APIs and a canonical WASM ABI. The current implementations of this specification include: • Wasienv[1], which compiles Swift and C++ to WASM conforming to the WASI standard. • wasi-sdk[11], a lower-level compiler for WASI, requiring frontend implementation for specific languages. • wasmtime[6], a WASM runtime targeting the WASI specification. • wasix[22], a compatibility layer to make WASI POSIX-compliant when built with the WASI SDK. • wasm32-wasi[4], a frontend for Rust using WASI. WASI, with its component model, appears to have the most thoroughly documented ABI.
GoJS	Limited information is available, but it appears that the TinyGo compiler [3] targets WASM.

Table 1: Major WASM Compiler Toolchains and Their Descriptions

To talk about the parameter passing strategy we have to first talk about how types are implemented to begin with. Usually the choice of parameter-passing is closely tied to the underlying execution method of a language. Managed languages and especially high level ones generally make the choice of implementing types through boxing which in turn means they implement the call by sharing parameter passing strategy; which means that types which are cheap to copy are passed by value (boxed or not) and types which arent are passed in a way where reassignments do not affect the caller but mutations are reflected in the caller. Languages generally termed ‘safe’ which implement some form of memory usually implement both call by value and call by reference leaving it up to the user to decide which to use for what type through certain operator choices. Finally languages which do allow memory management as a native part also usually expose pass by address where you pass a pointer which you can then dereference to use in the calling function.

A common pattern we can spot with the design of languages is the approach that primitives; regardless of implementation; are copied, and usually user-defined types or just anything which would be unreasonable to copy can either be passed by reference by the programmer or is implicitly done so as the implemented parameter-passing strategy. One notable strategy which has gained particular notoriety in recent years in the case of pass by value is if you move the type by copying or by destroying it. Rust; similar to most other languages copies small stack allocated types; but in the case of anything heap allocated as a part of the rust borrow checker such values are destructively moved. When we talk about a move as opposed to a copy the simple idea is usually just that as opposed to copying heap allocated data when we pass something by value we instead simply; through some mechanism; invalidate the source object and create a new object which points to this underlying heap allocated data. The distinction being that we in essence only ever create a new owner for the heap data as opposed to creating a new owner with their own copy of the data.

Currently the two prominent implementations of these dynamics come from C++ with move constructors and Rust with its borrow checker. The main distinguishing feature of these implementations is that in principle with C++ the moved-from object is nullified but left in a valid state such that a destructor can be called on it

when it goes out of scope, whereas in Rust the old moved-from object is in an invalid state and would raise errors if used after a move, Swift has similarly implemented destructive move since v5.9 with the introduction of the consume operator. How long a particular variable exists from the point of creation to it being either destructively or non-destructively moved is generally referred to as the variables lifetime. As Rust has destructive move semantics as a central feature of its borrow-checker and does not use runtime Garbage Collection (GC) it relies on either implicit (via elision) or explicit lifetime annotations as a means of conveying how long a specific variable persists for usage before being destroyed or dropped. To then briefly summarize, we end up with essentially 3 variants of how languages pass parameters, we either copy, pass some type of reference (either with shared semantics or not), or we give ownership.

A good question then becomes how do we effectively implement parameter-passing in an interoperability setting. The most obvious starting point would be that; just as we want for programming languages; types which are cheap to copy should be copied. Most languages do this by defining usually some set of ABI (usually C) compatible primitive datatypes. As most languages implement a decent set of primitives in a roughly similar fashion passing these by value across a shared ABI is generally not too complex and usually forms the initial implementation of most FFI based interop systems. Once we move past basic stack allocated primitives we start to get into the realm of some complexity. A common type introduced to new programmers after primitives is the struct. More formally a struct can also be classified as a product type [17] with the inclusion of labels for access fields. More concretely we then call these labelled implementations of product types record types or more commonly in the context of Object Oriented Programming (OOP) classes, objects or the aforementioned structs.

As FFI based interop generally concerns some low level intermediary an important question which has to be asked is how do different languages represent object properties at a low level. Enumerating all the different object representation models for all languages would be somewhat out of scope and meaningless so I will simply describe the abstract approaches which are generally taken. On a high level we can distinguish broadly between two types

of objects; direct objects and indirect objects. Direct objects here refers to objects which are represented directly in memory and whose property access usually translate to basic pointer offset computations. Direct objects usually exist in most statically typed, AOT compiled languages generally denoted with the keyword `struct` or `class` as a convention. When it comes to bridging direct objects the aim is firstly that property accesses resolve to the same memory location and that the type of the property, or more specifically the underlying memory has some analogue in the target language.

In cases where the memory does not align we start to encounter different implementation approaches one can take. The most trivial approach would be to simply expose property getters across the FFI boundary and then handle the property types as necessary. Though if we take this approach at face value its simply not feasible especially for large scale API's in some foreign language we wish to use in a different target.

2.2.2 Memory management. A critical low-level consideration in the design of cross-language systems is how we deal with memory management. There are two important considerations we have to make here, how long a specific variable should stay accessible, commonly known as the lifetime for a variable, and how we should deal with allocation, deallocation and access for different types across the language boundary. Naturally in the implementation of languages these concepts are strongly related as we generally want an allocation and deallocation to correspond with the expected lifetime so as not to persist unused memory. Generally speaking as a byproduct of the underlying platforms used to develop languages we usually end up with statically typed languages often relying more on manual control of memory and dynamically typed languages almost always implementing some form of runtime garbage collection mechanism. Though with time we have seen that various popular statically typed AOT compiled languages have various approaches to limit the application of low-level memory control due to its notorious risk and often problematic usage, examples include the Resource Acquisition is Initialization (RAII) paradigm most commonly associated with how C++ defined the lifetime behavior for its class implementation which is usually tied to their scope, Automatic Reference Counting which is referencing counting clang extension used in Objective-C and Swift, the Garbage Collector implementation which is bundled as the runtime for Go programs, the borrow checker in Rust which either determines implicit lifetimes at compile time or uses explicit annotations from the programmer to determine when to deallocate an object in addition to being based strictly on destructive move semantics ensuring unique ownership of objects.

When we then look into the cross-language dynamics of different memory management systems there are a few observations to make. Despite the quite varied implementations that define memory management strategies we can create two broad categories, languages which manage memory primarily through static compile time constructs, and languages which employ runtime implementations of garbage collection. Now of note here is that while theoretically any statically typed AOT language could have a garbage collector historically there has been a preference to rely on management abstractions which more closely model the low level performer

control of memory but yield enough safety and ease of use to provide a functional tradeoff. The question which we can then raise is how do we reconcile such systems. Here we can explore two key areas, firstly how can we bridge systems which employ very different strategies of memory management, and then how can we bridge types which are implemented with specific mutation properties and levels of indirection in mind.

2.2.3 Memory: AOT compiled cross-language systems. An important example to look at of statically typed cross-language systems are the different attempts at bridging C++ and Rust. C++ and Rust have been mentioned a fair bit in this paper already but for good reason, a considerable amount of software has been written in C++ but as new languages emerged it became quite evident that despite the iterative improvements to the memory safety of C++ its hard match the deterministic safety of the Rust borrow checker. Yet as mentioned before, despite these languages roughly falling into the same category of languages; with both being aimed at high performance through granular control, static typing and minimal overhead inducing abstractions; they employ some considerably different memory management models. Despite these differences one thing we can observe, not just in how C++ and Rust have evolved over time but also how languages more broadly evolve is that they often either through a deliberate effort or just through natural changes implement common features. C++ adopted the aforementioned destructive move semantics, albeit implemented in a slightly different manner to Rusts. Additionally there have been various approaches, implementations and proposals to introduce Rust-style lifetime annotations into clang to get similar benefits in term of statically derivable safety guarantees about code. Where we then commonly see interoperability layers come in is in their translation and mapping of these design approaches in a manner consistent with what the programmer would expect. In the case of crubit[12]; Google's glue code generator for Rust and C++; we can see this take shape in the form of the application of clangs lifetime bounds to preform static analysis on C++ code headers to describe lifetimes in a language agnostic manner such that C++ APIs can be used on the Rust sided end with the lifetime properties of exposed methods sufficiently mapped.

Something which becomes more complex to bridge are the different implementations of move semantics, despite their abstractly similar nature the aforementioned key difference of the C++ moved-from object needing to remain in a valid state while the Rust analogue does not leads us to the fact that we need a solution in our host language (Rust) such that we do not break the expected behavior of either Rust or C++. The attempts at solutions here have primarily relied on the use of rusts pin feature, the most important property of which is that data which is pinned remains in a fixed memory location, thus ensuring that the C++ side invariant of validity till destruction is met. One other notable consideration is the notion of things being trivially movable. Something being trivially movable implies that the compiler can statically preform a move operation; meaning that it knows implicitly how to change the ownership of an object, in Rust all objects are trivially movable, in C++ this is not the case. An example of C++ objects which are not trivially movable are Polymorphic types, which is to say classes which define virtual functions that are declared in inherited

classes. As these functions rely on v-tables to work properly there is no trivial implementation which could safely define a move. The common way to deal with this discrepancy at least in the context of C++ and Rust is to simply limit interoperability to types which have sufficiently close analogues in the other language such that their move semantics sufficiently overlap as to limit the complexity of interoperability layers. Now despite these differences we can still observe that due to the languages existing and evolving in a somewhat similar domain, under a well defined set of constraints we can create systems which sufficiently bridge functionality. Furthermore, despite the constraints currently placed on most interoperability systems there is research and practical work aimed at widening these constraints. A natural aspect of language evolution especially when talking about a languages type system is the goal of creating a language which allows for systems that are well defined and behave deterministically under clear constraints but also exhibit enough features to be sufficiently expressive for the domain they are usually applied in. The point of this being as languages have become and will become increasingly better at translating ideas within their domain into concrete programs so too is there a desire within the context of cross-language systems to bridge these dynamics such that we aren't as limited by the minimal safe common subset when working through interoperability layers.

2.2.4 Memory: JIT compiled cross-language systems. Languages which rely primarily on runtime GC implementations, namely JIT compiled languages exhibit certain unique distinctions in the context of cross-language systems. The nature of these distinctions stems primarily from the fact that JIT compilers commonly insert extensive metadata into types such that we have full reflection capabilities. Reflection as opposed to introspection refers to the ability to not only introspect or view the properties of type itself (as opposed to just being able to access them) but also manipulate said properties at runtime again through means aside from the direct access. Statically typed languages have adopted minimal forms of this usually as a means of expressing polymorphic types through either inserted metadata like v-tables variations in C++ and Rust or externally passed metadata commonly known as dictionary passing used in languages like Swift and Haskell. It should be noted though that due to the varied benefits that can come from having full reflection capabilities languages like C++ and Rust have made concerted efforts to introduce such features into the language. Reflection specifically the mutation and observability of types through the provided metadata is then naturally what is often used to implement rather expressive and performant interoperability layers. The benefit of applying reflection here comes from the fact that we in essence already have built in type wrappers, by which I mean that the metadata and exposed methods to modify types is in essence what often has to be custom built for statically typed interop. layers in the form of heading parsers and wrapper types for things we exposing as references across the language boundary. Bringing this all back to the point of memory management, we can see that as opposed to the need to reconcile different systems of memory management in the case of Rust and C++, in this instance we can use the metadata provided and property modifiers provided by the opposing language to expose an idiomatic API front-end. The back-end then uses some common denominator bridge, such as a CFFI-API, to

perform mutations without ever having to deal with the potential complexities of reconciling different GC implementations. We generally implement bridging mechanisms in a manner such that we minimize the level of memory moves occurring across the language boundaries. Instead, we work at a higher level of abstraction with cross-language calls being dedicated to describing primarily the operations we wish to perform on the data as opposed to the data itself.

2.2.5 Memory: AOT ↔ JIT cross-language systems. The third case we will look at is the instance where we want to bridge an AOT compiled language with a JIT compiled one. When talking about this instance in the context of memory management we see some fundamental commonalities with either of the above cases, as with communication between AOT languages and JIT'd languages we still are fundamentally reliant on some common denominator for the backend communication design, where this variant differentiates itself is in the ways we reconcile and utilize the different memory management approaches. As elaborated before we often see it that with JIT compiled languages we have metadata giving us reflective capabilities, an additional aspect of this is that especially in dynamically typed JIT compiled languages the focus on abstraction and dynamism in their design leads to the addition of having good metaprogramming tooling. Similarly to the case of two JIT compiled languages the presence of this metaprogramming tooling is then what allows us to usually have a highly generic and extensible interface for the language offering it. The same cannot be said for the AOT sided portion of this equation. The primary issue stems from the fact that if our AOT language does lack reflective capabilities to avoid copying we must fallback to the more rudimentary approach of copying over our data. While the difficulty in implementing a copy across the language boundary here is mitigated through the application of exposed metaprogramming techniques to construct the mirrored types, there are two rather painful issues we run into

- (1) When working with values from the JIT compiled language we often have to manually take control of the GC.
- (2) In the case of AOT compiled languages we often lack the reflective capabilities to bridge more complex types.

The first issue; and the one which will be focused on here; stems from the fact that to have the desired idiomatic usage of an object coming from another language we have the aforementioned two options. We either create a wrapper for a reference type or we copy over the object and map the data to the closest analogue type that exists in the other language. Now the former option is a somewhat natural choice when we can easily write idiomatic wrapper types using the abstractions provided by reflection, if we do not have these provided abstractions the resulting wrapper types can often be functional albeit limiting and therefore not very idiomatic to use. Going with the second option of copying we can usually define a set of constrained well behaved conversion rules for generating idiomatic types coming from the JIT compiled language, though at a performance cost. Though in both approaches, reference type wrappers and copies we have the issue that data operating outside of the scope of a garbage collector is inherently not accounted for by it, thus we must manually instruct the GC to preserve any data used outside of its overview. Something to observe here is that this

is somewhat akin to the issue we faced in C++ and Rust due to the nature of the disconnected ways we were bridging the data. Currently most implementations which bridge a runtime GC language with a non-GC language (i.e. one that uses reference counting, borrow checking etc.) utilize manual calls to an exposed GC API to replicate the implicit behavior it would exhibit where we operating within its domain. Though there are means of minimizing the extent to which a programmer has to manually make these calls. One example being the Jluna library for interop between C++ and Julia, its approach is to implement generic methods for boxing and unboxing (moving data between the languages) that implicitly tell the GC how to behave such that we have a constrained interface for which we can yield idiomatic C++ types without having to care about what the Julia GC does. The notable key point again here being "constrained" meaning that behaviors which go beyond the described constraints for what can be implicitly bridged given the libraries design the programmer must fall back to manual control of the GC. A potentially interesting direction to explore would be the application of implicit (through parsing and static analysis) and explicit lifetime annotations, specifically the addition of a reflective build step which can parse the annotations and either at compile time or runtime insert the necessary GC calls to preserve the liveness of a variable the GC itself cant manage. While this is already being explored in the case of crubit through the use of an external parser over annotated C++ headers it would be interesting to see the implementation of such a mechanism compile time or runtime type reflection as a part of the language itself and not external tooling.

As a final note on memory management it would make sense to discuss some of the different ways and approaches for dealing with the low level models of types and different mutability characteristics of parameter passing strategies. Throughout this paper, the term 'idiomatically' is mentioned at various points. When we talk about using a language idiomatically, we mean using it in a way that aligns with its intended design and application, as is characteristic of the language. From the perspective of a programmer we can break this down into 3 somewhat broad categories

- (1) *Liveness*: Any sort of memory used by the programmer should stay accessible corresponding to the defined conventions of the language we are using.
- (2) *Utility*: Features of a particular language should be usable in ways and conventions we expect them to be used in.
- (3) *Performance*: The performance both in terms of time and memory should be a function of the particular language and the degree to which a piece of code aims to be performant.

While these categories might not encapsulate the entire notion of idiomatic usage they do cover significant ground when we constrain ourselves to cross-language programming. Previously we talked about liveness in the context of runtime and compilation systems, which took the more top down perspective, in the following I will try to elaborate more from the other angle, namely the more concrete perspective. While there are various ways in which one could categorize the memory models of types and levels of granular detail you could go into I will use the convention adopted by Egor Zhdan [28], this being reference types and value types. The former being types with some level of indirection and the latter being ones

without. Having talked a bit about value types before I will only mention them briefly. As mentioned previously there is the quite reasonable expectation that in a cross-language context the only things treated as value types are those which are cheap to copy, which usually ends up being some overlapping set of primitives between the languages. Reference types are those which usually form the bulk of the discussion when it comes to implementation approaches. For the mutability characteristics of reference types similarly to value types we would expect similar dynamics in a cross language setting as in our host language. Thus if we do pass pointers, across the language boundary either directly or through wrappers forming another layer of indirection, in mutations in the target or guest language should be reflected in the source language if the host implements pass/call by address. The more complex case is when language which implements call by sharing for reference types calls out to a language which doesn't implement the same dynamics. Call by sharing simply put just means we create copies of references when passing them to other functions, so in terms of mutability characteristics for properties as we do with with regular call by address (passing a pointer) but with the added aspect that assignments do not mutate the original memory location. If we would want call by sharing² to function in a cross language context with a language that does not implement these types of parameter passing dynamics it would mean more or less re-implementing these dynamics by having wrapper types in our target language which behave like the corresponding defined value references in the host language. The somewhat non-traditional nature of this convention means that it could be considered what you could call a non-goal, in that despite it contributing to the idiomatic usage of the host language its not worth the tradeoff in implementing complexity, especially as our host language evolves and specifications change. Finally two notable types to mention which is often of particular importance in an interop setting are callbacks and closures. Callbacks, when able to be passed across a language boundary, offer a beneficial feature whereby a programmer can treat a call into another language as a first-class function which provides some nice flexibility when working with foreign function calls.

In a somewhat similar fashion closures; which are functions that capture some lexical scope of an outer function; can provide certain unique flexibilities in their application. Unfortunately just as first-class functions offer greater flexibility in design choices they similarly compound the complexity in the design of interop systems. Callbacks are usually the most basic form in which most programmers use first-class functions; if we focus on the case of a callback which does not capture scope (i.e. is not a closure); there are still a few key points to consider. Firstly we have the somewhat intuitive point that similar to normal operations across the language boundary we have to ensure that things behave idiomatically in our host language, which is to say that we must ensure that any thing returned from the callback is either wrapped correctly prior to its usage. Another point to consider is what if something goes wrong in the callback, as languages can have different (or no implicit) ways of handling exceptions this must be taken into account. I could

²As is often the case when it comes to naming things there is some debate in regards to the term call by sharing as its just treating a reference as a value type, but I feel this creates a unique set of properties that it warrants special consideration in the context discussed.

enumerate various other considerations for the case of callbacks but ultimately alot of them reduce to a similar fact mentioned earlier that if we want safe interop there must be well defined constraints or broadly speaking systems to prevent misuse or monitor for unpredictable behavior. An interesting development by in regards to safe memory dynamics in FFI based interop was the development by Turcotte et. al. [24] in their application of dynamic taint analysis in their implementation of Poseidon Lua, a variant of typed Lua with an integrated C-FFI in the form of a merged type system between Lua and C and taint analysis as a means of distinguishing between memory influenced by the non-deterministic nature of C and "safe" memory from the Lua side.

Then we have the case of closures, these as already mentioned compound the issues that we get with callbacks by the fact that they capture things within their lexical environment. The manner in which closures capture variables in a lexical scope depends on the languages implementation of binding. We can broadly classify binding into two categories, when it occurred this being compile-time or run-time binding [14], and how it occurs, either using value based semantics or location based semantics. There are more nuanced differences in implementations that should be considered but for the sake of context here we will limit ourselves to a more superficial overview. To elaborate on how these categories relate we can construct a table

Binding Type	Compile-Time	Runtime
Value	Compiler replaces the variable with the value.	Value is computed at runtime and assigned.
Location	Compiler assigns a fixed memory address	Memory is allocated dynamically at runtime.

Table 2: Compile-Time vs. Runtime Binding

While this is somewhat of simplification it does enumerate a few of the key scenarios that have to be taken account about variables closures capture when we think about dynamics across the language boundary. More broadly speaking a lot of the dynamics come down to a point we mentioned earlier being that we must consider how the memory is being managed, in this case, within the captured scope. A mostly safe instance is the case where we have compile time value based binding, as this usually just translates into inlined values, thus is largely excluded from memory management systems as they are usually more focused on heap allocation, or more abstractly speaking bindings with some level of indirection. Which brings us to the more complex case namely languages which implement location based binding, in other words languages which bind a memory location to a variable in some lexical scope which creates a level of indirection to some underlying piece of data. In this instance we must consider the same case as we did earlier with either implement implicit control of the GC to preserve memory via the binding generator, or explicit control either via annotations such as the aforementioned lifetime annotations we have with Rust or as a feature with clang, or explicit preservation through features like memory pinning or using the GC API to override default behaviors. Again its a fair point to question in an applied context if something like this is worth implementing as opposed to adding a static or runtime constraint to prevent this all-together. While we have listed

here some mostly safe case and approaches for tackling the more complex instances in practice without clear formal specifications of language dynamics there is still considerable difficulty in create safe interop for scenarios where the level of determinism is harder to accurately determine. Certain languages such as Swift especially in its generics system [18] have taken great strides in creating a clear specification of behavior, but its nonetheless still difficult to create effectively safe bridges especially when we consider language features in which a semantic mismatch is considerably harder to define clear bounds of when we look at all the varied use cases of a specific feature.

2.2.6 Binding generator implementations. Having established a baseline for the low-level dynamics we can move onto some of the more high-level question when it comes to the design of binding generators. As is the case when we talk about most kinds of software we can broadly distinguish two important categories, the low-level and high-level aspects of our implementation. A very popular field in which this division is most commonly mentioned is the field of web development where we have a back-end usually running server sided code such as one which handles API requests and a front-end which deals with client actions such as making requests and rendering information. The division of back-end naturally extends to various other implementations of software systems, in the case of binding generators our back-end comes in the form of the low-level platform based considerations we have to make i.e. reconciling different memory layouts and execution models, the high-level or "front-end" is then what we actually expose to the programmer. A more apt descriptor of a binding generator front-end would be the API it exposes. An important distinction here is that when we talk about glue code generators there are two relevant API's:

- (1) *Generator API*: The generator itself, which includes the collection of methods and classes it exposes as features.
- (2) *Bridging API*: The collection of types this generator is capable of bridging into different languages, often related to bridging an API for a library or framework written in another language.

In the following I will focus primarily on (1) as its difficult to asses (2) in any meaningful fashion within the scope of this paper. It would make sense to establish some semantic clarity before we move on. When I talk about an interoperability layer this refers to the broadest possible descriptor of any software system which enables interoperability between languages which encompasses glue code, binding generators, foreign function interfaces, etc. The concrete piece of code which actually enables interop is usually a conjunction of a languages foreign function interface standard on which there are generally libraries which form abstraction proxies to enable easier and more flexible usage over having to deal with the more low level constructs. Binding generators as the name implies are pieces of software which generate a language binding, which is to say they are code generators for automating the creation of glue code. Having established the terminology now we can move on to the key question of how do these generators tend to actually produces the desired glue code ?

Abstraction once again becomes a very central aspect to how different implementations tackle the problem of generating the desired glue code. Naturally it might become clear that if we want

to generate bindings for an API across a language boundary we need some mechanism to actually understand both what this API expects as inputs and gives back as outputs. Expressing this expectation more formally there are 4 important stages we can define

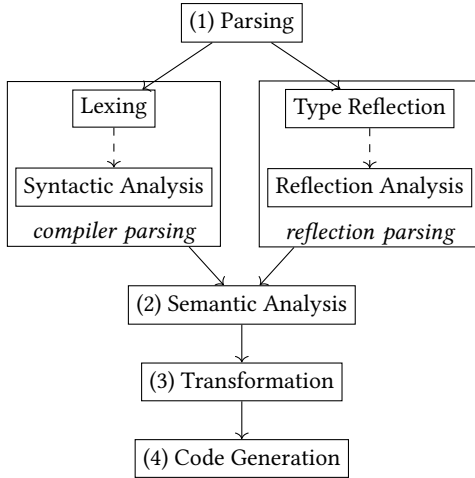


Figure 1: A high-level overview of the stages in a binding generator.

- (1) *Parsing*: We need some means of turning the API (i.e. our collections of types) into a malleable format.
- (2) *Semantic analysis*: Once we have our format, we need some way of understanding it to ensure we create a safe bridge.
- (3) *Transformation*: Once we have developed an understanding of the API, we need to define constraints to not expose functionality which would be unsafe.
- (4) *(Glue) code generation*: Finally, once we have defined the necessary constraints, we need to actually generate the necessary bindings to use the language bindings in our bridged language.

2.2.7 Binding implementation: Compiler based parsing. As these are dependent processes one of the most important implementation considerations becomes the choice of how we parse the API we wish to expose to the other language. An observation people familiar with compilers might have already made would be the similarity here to compilation processes of programming languages. The similarity to standard compilation leads into the first approach various glue code generators use for parsing, namely standard lexical and syntactic analysis based on a languages defined tokens and grammar. Despite superficially similar approaches being employed to front-end compiler design; as one approach of parsing; there is a common design paradigm of creating domain specific restricted parsers that naturally focus predominantly on parsing headers / type signatures as its ultimately not the entire language itself we aim to bridge, simply the interfaces this would make working with language parsers rather unwieldy as we would essentially have to extract only a minimal subset of the high level IR which actually refers to the information necessary for the glue code generation. Despite this the swift compiler team did implement the header parsing process utilizing the clang AST parser, though looking at some

of the most popular glue code generators this approach appears to be an outlier.

2.2.8 Binding implementation: Reflection based parsing. The other approach we can observe for gathering the necessary information about the types we aim to bridge is to utilize code reflection. I want to clarify that, as with most design considerations discussed in this paper, the wide variety of implementations means that the approaches analyzed here are not mutually exclusive. While I discuss them separately, real-world implementations may combine multiple approaches. My goal is simply to assess different attributes of these implementations in a way that made the most sense given the constraints of the ones I examined. Code reflection, as opposed to parsing through lexing and syntactic analysis, is a much more language specific implementation choice as the glue code generator is now integrated with the languages runtime as opposed to existing aside it as an addition build step for example. As reflection is a primarily runtime based process it allows for certain unique approaches for generating glue code as we can utilize runtime information though comes with its fare share of drawbacks. Unlike the more traditional compiler based parsing code reflection is naturally only present in a subset of languages as its not an intrinsic property of a language rather a deliberate runtime implementation feature of the languages design. Furthermore, as reflection, more concretely type metadata, implies overhead in conjunction with its predominantly runtime based nature we have the fact that many languages arent even designed to accomodate reflection. Finally we have the fact that reflection systems can potentially provide not enough or just not the right information necessary, abstractly speaking we run into a similar issue of using a language AST in that we first need to parse the format into our domain specific usage as opposed to just developing a domain specific parsing solution from the get-go. Nonetheless as I will elaborate in the binding generation section there are some very interesting results that we get from the dynamism that comes with applying code reflection in this situation.

2.2.9 Binding implementation: Manual specification. Manual specification is the final approach for how to generate bindings. Complete manual specification, by which I mean using very low level FFI libraries is a largely outdated approach for most languages as by now even for the case of manual specification there exist means by which extreme low level specifications are abstracted by utilizing different code generation processes, a simple example being the use of macros to expand binding specifications into the more low level format which we can pass across the language boundary. One area in which we see manual specification often occur even with binding generator tools is in making certain more nuanced modifications to the generated code or the generation process in instances where the generator tools yield certain undesirable formats in the cross language setting.

2.2.10 Binding generation. Now that we have established some of the important low level considerations in the creation of binding generators, and furthermore how we manage to represent certain type properties through things like type metadata representation models we can move onto the final question of how and when to actually generate the bindings. Abstractly we can take a similar approach as we did when discussing some of the previous topics

by looking firstly at the simpler case of overlap in certain design choices enabling an easier interop system and then the more complex scenario of languages which more substantial differences in design and implementation.

2.2.11 Binding generation: Similar languages. Firstly we can assess the case of similar languages. When we use the word similar here we mean similar in their implementation of specifically the type system and the underlying associated memory models for said types. It was already noted previously that commonly for interop implementations primitives, especially those in unmanaged languages; in other words primitives which aren't boxed; can often be trivially moved across the language boundary. We can see the same notion extend to record types for languages which represent these in roughly similar fashions, an instance of this would be moving structs between rust and C++ with crubit [12] in which there is little modification applied as these types have a fundamentally similar layout given the ideological overlap in both languages being aimed at performance and hence limited runtime dynamism in turn leading to similar straightforward implementations of types. Something we can notice here when viewing things from a higher level perspective is the parallels to the case of memory management. Languages which were designed with a greater focus on compile time programming and less-so runtime features tend to have a minimal set of trivial overlap in their type design which allows often for a very constrained albeit quite functional and performant bridge to be formed. Though the constraint here being that the lack of runtime considerations lead to attempts at bridging more complex types being relegated to certain build step features, often the bidirectional header and wrapper generation process through the aforementioned parsing. When viewing this from a practical perspective it makes sense given the general application if these languages, but it unfortunately means that if we want to bridge say different implementations of generics we often can't rely on runtime flexibility or introspection as these are naturally somewhat underdeveloped features in languages which understandably aim to mitigate runtime overhead of things like detailed type metadata.

2.2.12 Binding generation: Dissimilar languages. Having assessed the binding generation for the case of similar languages we can now move on to the case of dissimilar languages. Again, as we saw in the case of memory management, this leads to some interesting implementation features. To recount it's a somewhat natural desire when designing glue code layers to ensure that both languages can be idiomatically used, though this desire is unfortunately somewhat antithetical in the case where languages exhibit some fundamentally different design paradigms. The result of this tends to be classifiable into two design philosophies, connected and disconnected glue code layers. Connected glue code layers tend to form the most common type of implementation, or at least they usually make up the most dominant portion of glue code system implementations. What I mean with connected layers are systems of (partially or fully) bidirectional interoperability wrappers / bindings which expose the constrained set of features which can be safely bridged, often implement implicit assurances that would be expected from the language we are getting our binding from. This is a somewhat natural implementation as it provides a very explicit and safe proxy for interacting with foreign type information, the

downside naturally being that for fully idiomatic usage the programmer is left with implementing a conversion of the bridged type to an idiomatic type and ensuring the foreign language has everything safely managed. A natural question that one might have then is can we just get the full idiomatic type from the glue code? Here we have to introduce some level of nuance, in that to clarify again it's commonly the case that between two languages there is a set of types we can bridge idiomatically and a set which tend to be bridged via wrappers. A particularly though understandable rare implementation is the disconnected glue code layer, which is making any aspect of interoperability when talking strictly about type usage hidden to the programmer. Which is to say, instead of types which are not easily bridged being exposed via a managed wrapper we instead use the glue code layer to implicitly generate bidirectional idiomatic types such. While in theory this sounds quite nice it comes with various downsides, a large one being that as opposed to encoding constraints on types within wrappers we have to encode the constraints within the idiomatic generation process itself, which raises two main issues

- (1) To even generate an idiomatic type with constraints that we only bridge well define aspects a language needs to support such a generation process, the implementation of which can be complex.
- (2) It naturally leads to limitations as wrapper types can expose language specific functionality that might not be shared by the common abstract idiomatic subset.

2.2.13 Binding generation: Phase of generation. Now we can move onto the final question of when do we actually generate the language bindings. In a sense this question somewhat mirrors the case of when to preform code generation for the concept of generics, as abstractly it's a somewhat similar idea, in both cases we are in a sense generating instantiated representations of things like methods and classes based on a defined set of types, just in the case we are often generating these features as runtime wrappers, compile time cross-language headers, as opposed to instantiated generics. The 3 most relevant points at which bindings are generated are during pre-processing, compilation, or runtime. As is the case with most features discussed there aren't as clear delineations as some systems can and often do implement various of these aspects they are important to consider in their own right.

The most common form we see is pre-processor based binding generation, as the name implies in this instance bindings are generated generally prior to compilation and execution of the program. Pre-processor based binding generation provides certain benefits, namely that we can preform the actual compilation with the bindings usually generated as code, not only does this mean we don't have to deal with complex runtime based implementations but as a programmer we can also get the benefit of static type auto-completion as we can directly work with the generated headers and wrapper types, as opposed to the runtime case which often inherently affords less static information. The main drawback of this is that it often comes in the form of increasing the build complexity, as we add an additional step into the process for generating the required interoperability code.

Moving on we have static compiler-based generation, while less common we still see various instances of this, a common case being

larger compilers which have implemented various approaches to interop in an effort to more naturally work with multi-language environments between older more legacy based languages and newer ones. One notable example being the `swiftc` compiler, which has a version of `clang` embedded which allows for implicit header parsing; which is to say header parsing not using a separate build tool but a feature native to the compiler itself. While it does require some minimal setup to get such a setup properly the resultant disconnected glue code layer creates an effective environment to work with Swift and C++ or Swift and its predecessor Objective-C [23].

The final approach for when to generate bindings is runtime-based generation. It's been mentioned a few times now that languages designed with dynamism in mind tend to be more effective at bridging certain more complex features due to their commonly reflective features; we see a similar case for the situation of when to generate bindings. Despite the fact that with runtime based binding generation we naturally do have a greater overhead, we gain the key benefit of runtime type introspection, this allows for some unique features, an important among them being that it gives us the ability to say reflect over cross language generic API methods we are calling, and then generate concrete instantiations through recompilation that can be used in a cross language environment, essentially enabling cross-language generics by simulating the dynamic type instantiations which you can often lack in languages based on compile time monomorphism for generics. [maybe include a nice little diagram here] Two systems which implement such features are the `JavaCPP`, and `cppy` library which allow interop from Java to `cpp` and `python` to `cpp` respectively [2] [16].

3 IMPLEMENTATION

So to begin this exploration of the practical side language interoperability I had to first think of a piece of software I could use to test this idea. Before doing so I thought it would be important to further refine my idea beyond just wanting to test out my 'lego idea'. As said before the concept of rewriting code is by no means new, I don't believe it would be absurd by any means to suggest that most programmers at one point or another, while writing some piece of code, or even after the fact, recognized ways in which their solution can be improved. Often improvements or iterations on the original piece of code just naturally have to occur to fit the evolving goal for a given piece of software. So it would make sense that there should be some type of reason as to why we want to rewrite a specific piece of code, in our case performance would be a pretty natural choice. While most programming languages of course aim to be performant, respective to their application, due to differences in their design and execution methods they naturally exhibit differences.

Ok, so we want to choose a piece of software, preferably one with a performance-critical component, which we want to rewrite. Looking back at my previous projects one specifically came to mind, my implementation of a simple Ray Tracer in the Julia programming languages based on Peter Shirley's wonderful guide and C++ implementation [19][20]. My initial goal with this project was to learn Julia as I had heard and seen that if written in a very idiomatic manner it can exhibit performance characteristics close to that of

C while offering the dynamically-typed high-level simplicity of Python. Naturally, when benchmarking my final implementation my results were less than desirable. To elaborate on why my program exhibited these performance characteristics it makes sense to briefly go into the core of how Julia's code is run. There is a very nice quote on the Julia forums which I believe effectively conveys this.

in short, it's a reasonably good static compiler but a terrible JIT

The first response some people might have to this is confusion. Julia is a JIT-compiled language so how is its JIT compiler terrible. Well, this comes down in large part to how its JIT compilation works, the core of this is how it performs online partial evaluation (OPE) w.r.t. its type system. OPE w.r.t the type system in this instance refers to the process of, at runtime (online), Julia specializes (partially evaluates) methods for which it can infer the types of every local variable from the types of the arguments or any other constants; in this instance, a method is said to be 'type stable'. Specializing type stable methods simply means it compiles methods for the set of concrete types it can infer. Practically speaking this mechanism can also be seen as a form of monomorphizing a set of functions for which the JIT compiler can determine the concrete types. The reason then why Julia's JIT compiler could be called 'terrible' would be that it, currently at least, does not implement profile-guided optimization (PGO). The lack of PGO and its reliance on type stable code for speed means that there is a correlation between idiomatically written Julia and its performance. With this understanding, it might now be apparent why my Ray-Tracer performed so poorly, as I was not fully aware of this strong dependence and I simply wrote the code how I would write something in any other dynamically typed high-level language.

We've now established that we have a piece of performance-critical software written in a very non-idiomatic and by extension inefficient manner. Furthermore, we've seen that its execution method is not only different from ahead-of-time (AOT) compiled languages but also from other JIT-compiled languages by its unique approach to generating machine code. The conventional route, and quite frankly recommended route, one should go here would be to simply rewrite everything with Julia's idiomatic principles in mind, though this would lead to a rather short exploration, so to question the alternative, what if we rewrote the most performance-critical segment of our ray tracer in languages which have less of a reliance on a highly idiomatic programming style.

In the following I'll briefly start by giving an overview of the ray tracer itself and then how I approached integrating the two different languages Python and C++ into this system. I'll then also motivate or give some insights into the reasoning behind why I chose the binding generators that I did, and some of their internal architecture and how it relates to some of the things discussed previously. Once all this is established I will elaborate on the issue which I encountered as a byproduct of the design of this raytracer when working with the C++ rewrite. From this I will go into the nature of the problem and by extension the constraints that came with the design of the C++ $\leftrightarrow$ Python bridge `Juluna`. With this information I will explain how I came to my approach for a solution and some of the pitfalls I had in its design. Finally I will do some benchmarking

of my solution and compare and contrast this with the approach that PythonCall employs. Ending things on a discussion about the limitations and possible future directions.

3.1 Part 1: We want to do something

3.1.1 System Overview. As one of the objectives of this paper is not only to research the theoretical underpinnings of interoperability but also assess their viability in practice we can move on to describing how a lot of the topics previously discussed have relevance in practice. In the preamble the decision was made to use a ray-tracer implementation written in Julia as the main subject for this exploration. Before delving into the implementation of the actual cross language aspects it would make sense to give some motivation as to why a ray-tracer serves as a good subject. The decision, despite being initially somewhat ad-hoc, does present some potentially insightful attributes. Ray-tracers, especially more developed legacy implementations, are somewhat well-known for being complex pieces of software, not only due to their foundation in mathematical modelling but also their interconnectedness of various different systems to enable the actual image generation. The complexity of even somewhat minimal ray-tracer implementations; as in the case of the one I'm using, nicely presents a situation analogous enough to more complex systems giving potentially some broader insights into the effects of interop as opposed to some domain specific minimal example. Performance is another key aspect of ray-tracers, while they had their roots in drawing simple images nowadays ray-tracers or more broadly speaking physically based rendering (PBR) frameworks are used in real time in video-games, in turn creating a massive demand for high performance PBR systems, while the ray-tracer used in this paper only implements a very minimal subset it still relates directly to a domain in which performance is critical, again serving as an indicator of the viability of the applied interop techniques in performance demanding scenarios. Finally, despite the aforementioned complexity of ray-tracers they do tend to be very modular systems in some respects, this modularity should allow for an easier time separating predominantly logically distinct components and thus make the rewriting process easier. A high-level overview of the ray-tracer can be seen in Figure 2.

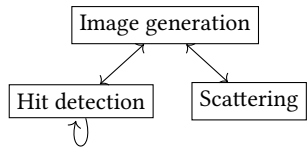


Figure 2: Ray-Tracer high level control flow

3.1.2 Implementation approach. Now that we've established our motivation for testing and have an implementation to work with, the next step is figuring out how to test it in practice. To isolate the most performance-impacting component, we profile the application, analyzing both runtime and memory usage before making informed judgments based on this data. Regarding language choices for rewriting, we are constrained to those with embedding APIs for Julia, but we still have several options. It makes sense to choose two languages with fundamentally different properties—Python

and C++—as they provide a broad spectrum of insights applicable to similar domains. Both languages have well-established interface libraries, making them practical choices for evaluation. To test the effectiveness of the component, we use benchmarking tools to analyze its performance in isolation, along with Julia-sided tools to assess the combined performance of the component, interface code, and any overhead introduced.

3.1.3 Choice of Binding Generators. Writing the components alone isn't enough if we can't integrate them into the larger system, so the next step was selecting an embedding API to attach our hit functions to the ray tracer implementation. For Python, the choice was straightforward—Python/JuliaCall [21], which builds on an earlier implementation and supports a wider range of conversions between the two languages. C++, however, presented a more interesting challenge. Unlike Python, which has trivial reflection mechanisms making object conversion straightforward, C++ requires more deliberate choices. The two main options were CxxWrap [15], which has a strong feature set but lacks user-friendliness, and Jluna [8], a newer and more user-friendly alternative that is missing some mapping mechanisms. I chose Jluna because my focus was on minimizing additional code and effort while creating usable components in another language. Jluna, leveraging modern and extensible C++ techniques, proved to be more future-proof and developer-friendly. This ties into a broader consideration: when rewriting a component in another language, if the results are positive, you ideally want minimal refactoring to integrate it into the larger system. This means choosing an API that (1) requires the least additional code and (2) minimizes the code needed to attach the component. One key takeaway from this process and something which might already be quite evident given the discussion in 2 is that working between Python and Julia is already posing significantly fewer challenges when searching for a binding generator due to the added feature set we get from both languages being designed with dynamism; and by extension runtime reflection; in mind.

3.1.4 Implementation setup. The implementation setup to test the component rewritten in Python and C++ can be seen visually in 3. The code for setting up the scene, which includes parsing the '.obj' files, that describe the vertices which bound all faces in scene, loading the faces into a bounding volume hierarchy (BVH), and managing the actual image generation by drawing the pixels is all uniformly done in Julia. The isolated component, namely the tree search and hit computation for a given ray cast are what is rewritten in C++ and Python, and then replaced in the Julia implementation depending from which language we initiate the ray-tracer. In both instances we redefine the hit functions such that the Julia code is redirected to the language specific variants which act as proxies to the actual host language code.

3.2 Part 2: We have a problem

3.2.1 The problem. Now while in theory the implementation here was straightforward a major issue I encountered was related to the constraints placed in the binding generator between Julia and C++. To explain the cause of this issue it makes sense to first go into what our implementation actually needs to do, or more specifically what we are bridging in our case. As the hit detection functions; the

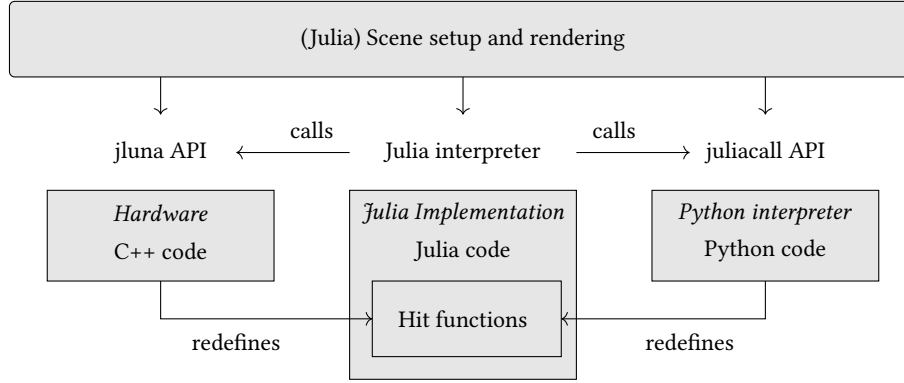


Figure 3: Cross-language implementation of the ray-tracer

functions which made up the BVH traversal; served as the nicest component to isolate due to their aforementioned logical separation from other aspects I chose them to serve as the bridge. Now broadly speaking the 3 main things needed when we are trying to do hit detection is our BVH which spatially structures the objects in our scene, the actual ray for which we are trying to compute a hit, and finally a hit record which stores the actual information from the hit (e.g. material, normal vector to the surface, etc.). Assessing the complexity of these types the simplest one is the ray at is just stores an object and direction, next we have the hit record; which despite being a struct; does have polymorphic attributes, namely the material of the object which the ray hit, finally we have the BVH tree which as implied in the name is a tree structure which adds another layer of complexity as we now have recursive polymorphic attributes with the left and right child being BVH nodes which subtypes that define the underlying type of the shape represent (e.g. Triangle, Sphere, etc.). Visually we can represent this as follows.

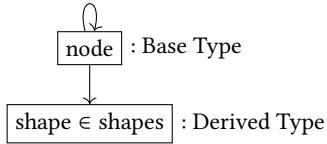


Figure 4: BVH Type Structure

In both of these instance the type of polymorphism we are talking about is subtyping polymorphism, while both C++ and Julia; abstractly speaking; implement similar conventions in terms of enabling subtyping polymorphism, namely some abstract type and a means of specifying a type hierarchy, there are considerable differences when it comes to the practical case of interop. A very distinction between the high level notions of interop and the low level practicalities is how types are actually represented, as we are still fundamentally working through the C-ABI or more accurately through an abstraction (Jluna) over the Julia C-API before we do anything we need to actually parse the underlying representation of Julia types. Pleasantly enough this process is elegantly implemented within Jluna for all types up to structs. The manner in which Jluna

does this is largely by using generics to recursively deconstruct the low level representation of a Julia type. To ensure constraints on the implementation, by which I mean rejecting types which it cannot parse for some C++ analogue the implementation utilizes constraints (the implementation being concepts in this case) on its generic deconstruction methods which ensure certain invariants hold that constrain the set of bridgeable types to only those specified by the programmer. One of these constraints was a static assertion which prevent bridging abstract types, in other words types which exhibited subtyping polymorphism, which brings us back to the issue I ran into. The question then becomes how does one solve this problem. To contrast this with the case of Python and Julia; somewhat expectedly; the story was a lot easier. JuliaCall, which is the library I chose for bridging between these two languages simply wraps Julia types with python objects that idiomatically mirror their Julia counterparts, which in turn means that we retain all the means to easily replicate the polymorphic behavior in Python. Trivially a straightforward solution to the C++ and Julia interop. would have been to simply rewrite the ray tracer such that the BVH and hit record represent everything in a non polymorphic format, though there a few reasons I was curious about trying to bridge these types.

- (1) *Challenging Common Notions*: It's a somewhat common notion when you look at the interoperability goals for bigger companies to simply constrain the set of bridgeable behaviors to minimal things. I was curious to challenge this notion; with the increasingly expansive type systems even mainstream languages implement, sooner or later, a hyper restrictive approach to cross-language systems will substantially limit libraries which apply more complex aspects of their type system in library design.
- (2) *Flexibility Discrepancy*: Furthermore, I wanted some clarity on to what extent the general discrepancy in flexibility between dynamic languages and statically compiled languages in the context of cross-language systems spoke to some basic nature of type systems and execution models or was more a function of certain implementation philosophies or limited applications of a language's features.

- (3) *Viability of Loosening Constraints*: I was curious to test the viability of loosening the constraints both in terms of implementation complexity of the system and the resulting performance effects that come from bridging these types.
- (4) *Feasibility Exploration*: I was generally curious about the feasibility as a whole, given that it's a lot rarer to see complex type bridging for cross-language systems involving statically typed languages, especially ones where the statically typed language is in focus (as opposed to relying heavily on the dynamic language). I felt this would potentially provide some interesting insights.

3.2.2 The more concrete considerations. With the broad idea in mind of what we want to achieve now and some motivation as to why it makes sense to establish a more concrete approach. Before we asses the details for how to extend the Jluna library it makes sense to first asses in detail how the library works currently. As was discussed throughout 2 if we think of C++ as our host language and Julia as the target then boxing would be the operation of representing a C++ type in Julia, and unboxing the reverse. In Jluna the boxing and unboxing have multiple steps, we can give a high level overview of it as follows. With a general breakdown of what each step entails.

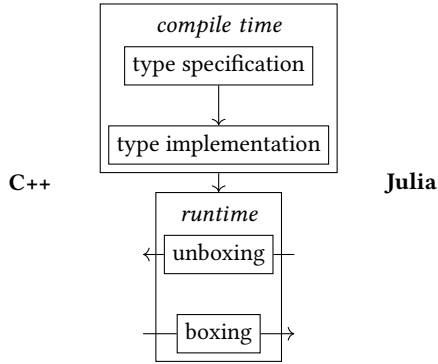


Figure 5: High-level overview of Jluna application

- (1) *Type specification*: For the type specification of object types (i.e. classes and structs) the programmer defines a set of getter and setter functions in addition to some other type attributes which are then encoded in a meta-type akin to meta-types generated via dynamic reflection mechanisms in dynamic languages.
- (2) *Type implementation*: Once this type is defined the programmer calls an implementation method which utilizes the meta-type to generate an idiomatically mirrored Julia type using its metaprogramming features which creates the corresponding type within the Julia runtime.
- (3) *Unboxing*: Unboxing is then applied to all arguments on methods which are exposed as cross-language via an implicit unboxing proxy which is generated through the specification of the function type signature in the Jluna framework.
- (4) *Boxing*: Conversely boxing is applied to all return values of the function which are then passed back to the host language, again as an implicit wrapper around the bridged function.

A first rather obvious question would be how do we implement our extension without having to massively rewrite the existing library. As mentioned before the library relies heavily on utilizing generics, more specifically template metaprogramming, which is an entirely compile time construct in C++. The focus on compile-time metaprogramming meant the type specification was naturally reliant on compile-time information, which in turn meant that the type specification was largely static. Given this emphasis on static type specification I wanted to see if it was possible to integrated a static polymorphic type specification into the existing framework; as opposed to a more runtime based process. Despite the somewhat antithetical nature of the two concepts; static compile-time type specification of a runtime polymorphic type; constrained to the paradigm in which the framework was designed, I was curious to see if this was a feasible extension. To then approach this in a structured manner I first had to understand abstractly the the different attributes of the BVH tree which I needed to encode, and from this gain an understanding what precisely needed to be extended. If we do a high level breakdown of how you might describe the meta attributes of the BVH tree we can decompose it as follows.

Attribute	Type	Description
Abstract base class	BVHNode	An abstract base class which represents the BVH tree
Abstracted attributes	BVHNode	The left and right child nodes which represent either inner tree nodes or leaves
Inheriting classes	shapes	The shapes which exist on the leaves of the BVH tree and all exist as derived types from the BVHNode class

Table 3: BVH Type Attributes

With this in mind we can make few key observations already. As stated before the library was designed to fully disallow abstract types, hence this would likely form the first step to our extension. Furthermore as this library was not designed with abstract types in mind it naturally did not implement a way to accomodate such types, or attributes of such types, in the type implementation (Julia instantiation), boxing or unboxing. From this there are a few interesting questions which arise as to how we proceed.

- (1) *How do we encode the abstract base class?* Given that the library was designed to fully disallow abstract types, the type specification needs to implement a mechanism to express subtyping relationships.
- (2) *What are the effects of this constraint on the implementation of the library as a whole?* More broadly this is a question of extensibility in library design often appears. One interesting aspect of Jluna specifically is its heavy usage of compile-time metaprogramming and highly generic type encodings as opposed to for example concrete wrappers or more traditional parsing approaches as a means of generating bindings. A natural question which then arises is how does the extensive application of these paradigms effect the ease of extensibility given the constraints placed on the initial implementation.
- (3) *Unboxing - How do we parse the abstract Julia type in C++?* Naturally as the nature of our problem is bidirectional we

need to reconcile how the low level BVH tree representation in Julia is parsed in C++.

- (4) *Boxing - How do we represent parse the abstract C++ type to create the corresponding Julia type?* Similarly we need to ensure that the abstract C++ type is not only implemented in Julia but that we also have a means of moving the C++ type to this corresponding implementation.
- (5) *Constraints - How do we loosen constraints while retaining safety?* Finally we need to make sure that we can not only represent arbitrary BVH trees but also that we establish a means of ensuring that the constraints placed on the type specification are loosened in a way that does not compromise the safety of the system. Which is to say that we should define a new set of constraints for what constitutes a valid type polymorphic type specification.

3.2.3 What is metaprogramming? Before getting into the actual code which implements this extension there are two points I feel are important to establish in more detail before we move on. I've mentioned a few times by now the use of compile-time metaprogramming in the Jluna library, and while I've given a high level overview of what this means I want to dedicate some time to explaining some of the aspects of this design paradigm in the original implementation and furthermore how these aspects were applied to address some of the questions which were raised in the process of developing the extension. We can start by giving a quick overview of templates in C++ and how they are used in the Jluna library.

```
libs/jluna/.src/usertype.inl
```

```
1 template<typename T>
2 bool Usertype<T>::is_implemented() {
3     return _implemented;
4 }
```

Listing 6: Jluna implemented query function

In 6 we see the most basic application of a template, and one which should be familiar to most programmers who have used generics before. In this instance for the set of specified types $T \in T$ the function will be instantiated for each type in the set. A few observations to make here is that the type T in this isolated instance is not bounded, which is to say that; ignoring `Usertype`; this function could be instantiated or specialized with any arbitrary single type. To allow for some level of restriction in the types which are permitted to specialize a template C++ implemented its version of constrained generics called concepts. Concepts define compile-time predicates which express a requirement on a template argument. We can see a basic application of this here.

```
libs/jluna/include/usertype.hpp
```

```
1 template<is<double> T>
2 unsafe::Value* box(T value) {
3     return jl_box_float64((double) value);
4 }
```

Listing 7: Jluna concept function

In this case the concept is used to call the most appropriate variant of the box function for the double type. What this enables is static method dispatch, in other words it uses constrained parametric polymorphism to emulate the behavior of dynamic dispatch as the constraints define a subtyping relationship. Naturally the language extends this idea to multiple type arguments which each can have their own set of constraints. A question which might arise from seeing this language definition is what if you want to define a template with a set of types of arbitrary length. Generics which accept an arbitrary set of types are known as variadic generics, or in the context of C++ variadic templates or template parameter packs. Furthermore the concepts can also be applied to these variadic templates. Once again we can see an example of this being applied in the Jluna library.

```
libs/jluna/include/proxy.hpp
```

```
1 template<is_boxable... Args_t>
2 Proxy safe_call(Args_t&&...);
```

Listing 8: Jluna variadic concept function

On a high level all that's happening in 8 is that we are checking the predicate that for each member of `Args_t`; which represents an arbitrary length set of function arguments; the type of this argument is boxable and hence transferrable across the language boundary. In this snippet we see two new pieces of notation introduced, namely `'...'` which is used to denote a variadic generic, and `'&&'` which in this represents an *rvalue reference* which is a reference that binds to an rvalue (temporary value) which is about to be destroyed enabling move semantics for all arguments passed to the function.

A final relevant construct worth mentioning is the template metafunction, a basic example which can be seen in listing 9.

We can see a basic example of a particularly relevant construct of template metaprogramming which enables the above mentioned below. There are various interpretations of this construct depending on what it's applied to, in the most direct case it's simply a generic struct which exposes its template arguments as statically bound attributes. An analogous variant of this idea of generic objects can be seen in the implementation of traits whose attributes are used to define shared behavior for some set of implemented types akin to instantiating a generic. Traits or interfaces in most languages can then be described as being value level abstractions in that they give us an abstract or shared view of what behavior some set of types might have. Moving to a higher level of abstraction we then come to the idea of type traits, which are traits that; as opposed to describing shared behavior; they describe a type itself. In the context of C++ we can see this being applied heavily on conjunction with constraints, as if we have a means of expressing certain properties of types it's not too far out there to then use said properties to define compile-time constraint predicates. Stated differently constraints in the context of C++ are simply type traits which utilize partial specialization to implement compile-time control flow which in turn enables compile-time predicates on types. The more broad terminology of this idea is a template metafunction, which is intended to express a function which operates in terms of template arguments, or just types, as opposed to values and 'returns' a type

or computation applied to this type. For completeness we should also address the fact that the example 9 has a non-type parameter. C++ allows for constant value-level parameters to be passed to templates, which can be used to define compile-time constants which should be applied in the template instantiation process. A more formal way of viewing this is that template metafunctions can be defined as dependent types.

```
src/cpp/example/mfuncex.cpp

1 template <typename Object_t, const char* name>
2 struct ObjectMeta {
3     using obj_t = const Object_t;
4     static constexpr const char* Name = name;
5 };
```

Listing 9: Template metafunction function

3.2.4 Binding generators as partial evaluators. While constrained generics are a somewhat standard feature in many different programming languages an interesting observation to make; even with just these basic constructs; is that we in essence have two different programs. The first being the program consisting of the source code the programmer interacts with, the second being the instantiated program which is generated at compile time given the statically defined type specifications. The code generation process by itself is a natural byproduct of how instantiating generics works, what's interesting to consider here is the extent to which we can use maximally utilize this code generation process when designing libraries. An important notion here is the idea of metaprogramming, which is the idea that, as opposed to writing programs directly, as we might in a more traditional sense, we write a program which generates a program; as the name implies working on a 'meta' or simply higher-level of control and abstraction. To be more specific we can introduce the concept of partial computation also known as partial evaluation. Partial computation, first introduced by Futamura [10], states that given a function f processing some known data k , and unknown data u we can generate an intermediate program f_k which is the result of computing everything in f with k and then computing $f_k(u)$ we can get a final result. Visually we can represent this as follows.

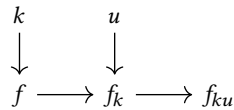


Figure 6: Partial Evaluation Process

Expressing this in more rigorous terms we can define a partial evaluator p_e as a function which maps a program p and some collection of statically known information I_s to a program p' called the "residual" via specialization, with p' then mapping the dynamic information I_d i.e. runtime information to a given output O . Formally we can express this as follows

$$p_e : \langle p, I_s \rangle \rightarrow p' : I_d \rightarrow O$$

Bringing this back to metaprogramming an observation which was noted by [26] is that this act of specializing based on the static binding of types to templates, is quite analogous to the process of partial evaluation. Something which gave further ground to this idea was the fact that templates allow non-type parameters, recursion and partial specialization. As the recursion implies self-reference and the combination of non-type parameters and partial specializations can be used to express a base case, we can define various control flow structures in the template or more broadly type level, something which was first discovered by [25]. Due to the somewhat accidental nature of the discovery template metaprogramming and the notoriously cumbersome syntax of C++ templates, the idea of applying this paradigm in any extreme length has largely been relegated to niche instances as only more recent versions of C++ have tried to really focus on expanding the feature set of templates.

Some important observations we can take away from this similarity between template metaprogramming and partial evaluation is that firstly we can view the process jluna defines to specify and implement types as a form of partial evaluation. Specifically we can consider jluna as a binding generator-generator which is to say that jluna represents a partial evaluator which takes the type specifications and the generic logic used for bidirectional transformations and defines a specialized binding generator which is then compiled. Expressed in formal terms if we consider jluna as a function G_{L_1, L_2}^1 which maps a set of types specifications T and a set of binding generator logic C to a binding generator C_T we can express this as follows.

$$G_{L_1, L_2}^1 : \langle T, C \rangle \rightarrow C_T \quad (1)$$

If we view G_{L_1, L_2}^1 as being an interpreter for a given type specification then we can see this as representing the first futamura projection. Abstractly speaking futamura projections express different stages of 'meta' computation, the first projection represents what was described above, a partial evaluator which specializes a program based on some known information. The second projection then represents the notion of specializing the specialized for the interpreter. In the context of binding generators this would be the process of specializing the binding generator for a particular set of languages. Formally we could define this specialized as a function G^2 which maps a pair of languages $\langle L_1, L_2 \rangle$ to a binding generator G_{L_1, L_2}^1 as follows.

$$G^2 : \langle L_1, L_2 \rangle \rightarrow G_{L_1, L_2}^1 \quad (2)$$

In practice implementing G^2 would require us to work at a higher level of abstraction, now instead of generic logic which is meant to apply to the set of types we wish to bridge we define the abstract attributes of a binding generator itself. What this means for example is that boxing and unboxing logic would now be defined as specializers methods given statically bound language specifications by the programmer whereby the logic would create concrete variants which functioned over some set of bridgable type specifications. Currently no binding generator I am aware of implements a full second futamura projection, an example which comes is a framework like Simplified Wrapper and Interface Generator (SWIG) [9] though while this tool technically accept arbitrary target language specifications it's host languages are limited to C and C++,

furthermore the tool does not specialize a binding generator for the languages instead it directly generates the bindings based on the parsed C/C++ headers.

The complexity of implementing a full second futamura projection is in large part a function on the degree to which the language specifications describe the required interoperability operations. In the most trivial case the language specifications would describe most of the underlying logic in which case G^2 would only have to ensure that the specification of this logic exists in a form which can be form which can be specialized. Conversely the most extreme case would require the specialized to implicitly derive or account for the entire interop logic by figuring out what would constitute the set of valid type specifications and the corresponding logic to bridge them. A reasonable middle ground would be to define a Domain Specific Language (DSL) which could transpile to a more complete language specification essentially serving as a proxy between the programmer and G^2 . Using a DSL as a means of abstracting away complexity has already been seen in the work of Zhang et. al. [29] which used a DSL to unify kernel programming. A notable feature of this DSL was that it is defined entirely within C++ by using template metaprogramming in addition to macro expansion. The nature of having a DSL embedded which is based on compile time constructs means that we can provide custom high level abstractions while reusing all existing language functionality and having intrinsically seamless interop with the host language.

An important thing to take away from the idea of thinking about binding generators as partial evaluators is that it we end up with a more well defined separation of concerns between different steps of the binding generation process which can offer up a more effective foundation for implementing future generators in a more uniform and preformant manner by isolating the statically known specifications from the actual implementation logic and viewing the implementation itself as a specialized over the specification as opposed to being a direct implementation of interop logic. The relevance here being that it supports the fact that any extensions made to the type specification should be done in a way that does not compromise the existing logic of the binding generator to ensure we maintain this quality of separation of concerns.

3.2.5 Approach to a solution. If we take this idea of viewing Jluna as an interpreter of a type specification grammar we can create a basic formalism which can help us in providing a more rigorous basis for our extension. First we establish the existing specification grammar and application rules, from which we can build a basic formal model of the relevant algorithms. From this we can expand the syntax and incorporate the new rules which would allow for bridging abstract types.

$$p_t \langle x, y \rangle = \lambda x. \lambda y. \lambda f. f \ x \ y \quad (3)$$

Expanding this to support typed pairs we introduce the notation from System F

$$p_t \langle x : \tau_x, y : \tau_y \rangle = \lambda x : \tau_x. \lambda y : \tau_y. \lambda f. f \ x \ y \quad (4)$$

Furthermore we define accessor functions for the pair as follows

$$\begin{aligned} (a_1 : p_t \rightarrow x) &\leftrightarrow (\lambda p. p \ (\lambda x. \lambda y. x)) \\ (a_2 : p_t \rightarrow y) &\leftrightarrow (\lambda p. p \ (\lambda x. \lambda y. y)) \end{aligned}$$

Now with this we have the notion of a idea of a field type defined but we still need to actually define how we can represent some collection of these fields. Informally we can describe a record type as being a collection of labels and corresponding types, with the instantiation associating values with these labels [7]. One drawback with this approach is that it does not express the nuance of what record types actually represent. In a more nuanced perspective we can think of record types as being dependent n-ary product types which are indexed by a set of labels. To break this down into different parts we can first consider n-ary product types.

Definition 3.1 (n-ary Product Type). An n-ary product type, generalizes the notion of a standard product type between two types to n types. Which is to say that it expresses the sets of all possible n-tuples of types. Formally given a set of types T we can define the n-ary product type as follows.

$$T_1 \times \dots \times T_n = \{(\tau_1, \dots, \tau_n) \mid \tau_i \in T_i. \forall i \in [1, n]\} \quad (5)$$

We can then consider a dependent type can be thought of as a type which can vary over a set of values, as we saw before with the case of non-type template parameters for C++.

Definition 3.2 (Dependent Type). Expressing this formally we can say that a dependent type is a type which is indexed by a set of values belonging to another type, formally this idea is represented as follows.

$$x : A \vdash B(x) : \text{type} \quad (6)$$

Which is to say that for some value of type A we can define a type $B(x)$.

A dependent type by itself only describes the family of types indexed by another type. A common example of a dependent type is if we consider A to be the set of natural numbers and B to be vector. The dependent type $B(x)$ then represents the family of types consisting of all vectors of length $x \in \mathcal{N}$. If we wish to express the collection of dependently typed function which assign each $x : A$ to an element of $B(x)$ we can express a dependent product type as being formed out of a type A and a dependent type $x : A \vdash B(x)$.

Definition 3.3 (Dependent product type). A dependent product type denoted $\prod_{x:A} B(x)$ is a type which represents the collection of all functions which assign each $x : A$ to an element of $B(x)$. We can represent the formation of this type using the following rule

$$\frac{\text{DEPENDENT-PRODUCT-FORMATION} \quad \Gamma, x : A \vdash B(x) : \text{type}}{\Gamma \vdash \prod_{x:A} B(x) : \text{type}}$$

For completeness we can demonstrate what application of a dependent product type looks like for a given input.

$$\frac{\text{DEPENDENT-PRODUCT-APPLICATION} \quad \Gamma \vdash f : \prod_{x:A} B(x) : \text{type} \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}$$

This should hopefully make the idea clear that once we actually apply some input we can see quite trivially the dependent nature that is being expressed here in the return type of the function f . From this it might already be clear how we can apply this to formalize the idea of indexing into a record type. To get the last piece of the puzzle we have to describe how indexing would work for an

arbitrary input. Which is to say we want to derive a λ -abstraction which contains a term of type $B(x)$ and a free variable $x : A$. We can express this as follows.

$$\frac{\text{DEPENDENT-PRODUCT-INDEXING} \quad \Gamma \vdash x : A \vdash b : B(x)}{\Gamma \vdash \lambda x. b : \Pi_{x:A} B(x)}$$

Finally we can express the trivial β -reduction rule applied to the λ -abstraction.

$$(\lambda x. b) a \rightarrow_{\beta} b[x := a] \quad (7)$$

With this we can now express the idea of a record type formally.

Definition 3.4 (Record Type). If we now assume that we have a set of labels defined by a type $L : \mathcal{U}$ and a family of types $T : L \rightarrow \mathcal{U}$. We can define a record type as being a dependent product type indexed by the set of labels, substituting the new notation in for the created λ -abstraction we can express the constructor for a record type as follows.

$$\Lambda L, T. \lambda x^L. \lambda \tau^{T(x)} : \prod_{x:L} T(x) \quad (8)$$

To show how this lambda function arises we can express the formation rule for dependent product types using more functional but equivalent semantics.

$$\frac{\text{RECORD-INDEXING-FORMATION} \quad \Gamma \vdash L : \text{type} \quad \Gamma, x : L \vdash T_x : \text{type}}{\Gamma \vdash (x : L) \rightarrow T_x : \text{type}}$$

$$\frac{\text{RECORD-TERM-INTRODUCTION} \quad \Gamma, l : L \vdash \tau : T_l}{\Gamma \vdash (l \rightarrow \tau) : (l : L) \rightarrow T_l}$$

From this it should be trivially clear how application then acts as just an instantiation of the record indexing process. Stated differently we can now see how on the type level a record represents an abstract mapping between a label set and type set, with the instantiation then representing the actual concrete mappings between labels of this set and values of the corresponding types.

To incorporate the idea of subtyping into our description we can describe a high level correspondence between the implementations of structural subtyping and nominal subtyping to the context of injectivity and surjectivity when viewed a dependent type theoretical lens. We will only superficially describe the idea fiber as developing the full categorical semantics would be out of the scope of this work.

Definition 3.5 (Fiber of a Morphism). The fiber of a morphism or function $f : A \rightarrow B$ over a point B is the collection of elements of A that are mapped by f to this point. Informally we can denote this as.

$$\text{fiber}(f, b) = \{a : A \mid f(a) = b\}$$

We can then move on to defining the idea of injectivity and surjectivity in the context of dependent types. From which we can form a connection between our formalism of record types and the idea of nominal and structural subtyping.

Definition 3.6 (Surjective Function). A function $f : A \rightarrow B$ is said to be surjective if for all terms $b : B$ the fiber of f over b is inhabited. Here all we mean to express is that for every element in the codomain there is some element in the domain which is mapped to it. On a high level we can think of this expressing the quantifier of existence or in the set theoretic sense ‘at least one’.

$$\text{isSurjection}(f) := \prod_{b:B} \parallel \text{fiber}(f, b) \parallel \quad (9)$$

The notation $\parallel \text{fiber}(f, b) \parallel$ is the propositional truncation of the type expressing the fiber of f over b . For our purposes its sufficient to understand this as expressing the predicate that the fiber of f over b is inhabited, which is to say that there exists some element or term in the domain which is mapped to b . In essence it simply states the aforementioned quality of existence.

Surjection has a general correspondence to the more concrete idea of nominal subtyping. Specifically we can think of it expressing the idea that if there exists some abstract class A then we will always have at least one concrete class B which is a subtype of A . An important observation to make is that surjection allows for us to map multiple elements; in our case subclasses; to a single element; the superclass, a byproduct of this quality is that we do not have the property of uniqueness. We can view this analogous to the fact that in nominally typed languages unless we preform a, usually runtime, downcast an object only has those properties which are defined in its superclass.

Definition 3.7 (Injective Function). A function $f : A \rightarrow B$ is said to be injective if for all terms $b : B$ the fiber of f over b is a subsingleton. By which we mean to express that the collection of elements A which are mapped to b consists of either a single element or no elements at all. Here again on a high level we can think of this expressing the quantifier of uniqueness or in the set theoretic sense ‘at most one’.

$$\text{isInjection}(f) := \prod_{b:B} \prod_{a:A} f(a) = b \quad (10)$$

Injection then has a correspondence to the idea of structural subtyping. Structural subtyping; often seen more in dynamically typed languages; relies on implicit structural comparisons to express the idea of substitutability. The key distinction to nominal subtyping being that we preserve the uniqueness quality of objects, in practice this idea is often leveraged in allowing for more flexible usages of existing implementations. Uniqueness of object mappings means that unlike in the case of nominal subtyping where we typically have to downcast to access the full set of properties of an object, in the case of structural subtyping excess properties are preserved and at least in theory can be accessed without any downcasting as there is a unique mapping back to the original object which preserves the structure.

Of course this is somewhat of an oversimplification as type systems don’t necessarily implement these ideas in mutually exclusive ways or even in the most direct way. Nonetheless for the applications of our extension we can use these ideas to develop at least a somewhat more precise language to discuss the steps taken to extend the Jluna library. To then summarize and elaborate on how we can proceed, we have now developed a basic way we can more precisely use to speak about record types; the primary focus of

our extension; furthermore we have a way of understanding how we can express subtyping in terms of the same theory we used to talk about record types, namely dependent type theory. We also more broadly have established that metaprogramming and binding generators for that matter can be understood as partial evaluators on some static specification. Using this groundwork we can build some key ideas which we can directly apply in the construction of our extension, these ideas or takeaways will be developed in the next section.

3.2.6 Application: Types of Record types. We can first look at the record type again, specifically lets deconstruct what we can consider the field type.

Definition 3.8 (Field Type). A field type is the dependent type which is indexed by the set of labels. We can interpret this as the function we formed in the record indexing formation

$$(x : L) \rightarrow T(x) \quad (11)$$

We can see that this field type is an abstract view over the more concrete mapping we think of as labels to values of a particular type. When we instantiate this type we specialize it to some concrete mapping of labels to value. Or more accurately a compiler or interpreter would specialize this type to a concrete mapping of labels to values. If we go back to the analogy between a binding generator as an interpreter and put ourselves in the shoes of this binding generator then a field type as shown above would be considered already specialized for some set of labels and types. A question which we must ask ourselves then, what is the type of this field type?.

Lets first consider again what a type is. From a high level set theoretical sense we can think of a type as characterizing a set of elements belonging to this type. Alternatively; as demonstrated by the Curry-Howard Isomorphism; we can understand types as being constructors which encode some set of propositions and terms of this type as being proofs of these propositions. A common informally shared aspect regardless of interpretation being that types in a sense are just abstract descriptions. To then get back to the question at hand we can say that the type of the field type is abstractly speaking simply a description of the field type itself. Just by observation we can create an informal description describing the properties of this type.

- (1) *Label*: We can confidently say that all fields clearly must have a label $l \in L$.
- (2) *Output*: We can also see that field types have some dependent output type $T(l)$.
- (3) *Record type*: Each field is also associated with a record T .

Informally a natural way we might then represent a constructor for this type could be as follows

A few notes we can make here is that firstly as labels are generally strings we can express them as being concrete value types bounded by strings. Concrete value types bounded by some more abstract type such as strings or integers are commonly also called literals and can be understood as a form of static narrowing of a type. Secondly as we are working on an aforementioned higher level of abstraction the types of the attributes of field types are descriptions of the

paper/code/constructorex.py

```
1 class FieldType[Label_t: str, Out_t, Rec_t]:
2     label: type[Literal[Label_t]]
3     out_t: type[Out_t]
4     rec_t: type[Rec_t]
```

Listing 10: Field type constructor pseudocode

underlying types which is expressed via the ‘type[T]’ notation representing the type of the type.

A key issue we find with this is that while we have described a kind of recipe for how we might represent the type of a field, currently the description in 10 does not actually tell us anything about how we are supposed to *use* a field. So we need to expand this description, the somewhat trivial question which then appears is what does it mean to use a field type? We can quite easily say this as being some operation to get a value from the field, and some way to set a value to the field. So it would be reasonable to think that we want to expand our description to include two functions, namely a getter and a setter. To be more rigorous we can express these functions as follows.

Definition 3.9 (Field Accessors AKA Lenses). We can describe the getter of a field as a function which takes a record of type T and returns the value of the field in the record defined by T_x . A setter conversly is a function which takes as an input a record and some value of type T_x and returns a new record with the value of the field set to this value.

$$\text{get} : T \rightarrow T_x \quad (12)$$

$$\text{set} : T \times T_x \rightarrow T \quad (13)$$

Unsurprisingly this notion of field accessors has already been extensively studied, in various contexts. A common first way lenses are described is through the idea of database transformations in terms of the view (get) and update (set) problem. Commonly this is then placed in a more genral context within functional programming to describe the aforementioned idea of just getting and setting values. The abstract idea of different forms of data accessors is generalized to the concept of *families of optics* of which lenses are the most well known part.

In our case for the case of field getters and setters its sufficient to constrain ourselves to the notion of lawful lenses.

Definition 3.10 (Lawful Lenses). Since we don’t want our getters and setters to just behave arbitrarily as that would be a bit absurd we can define a set of laws which we expect our lenses to follow. Specifically we can enumerate what is defined as a lawful or well-behaved lens. There are 3 main rules which characterize a lawful lens. If we consider a source S (analogous to our record type) and a view V (analogous to our field type) we can define the following laws. One note on notation is that different literature uses different notations for getting and setting, in this case we will use the notation

get and put.

$$\begin{aligned}
s : S \vdash \text{put}(\text{get}(s), s) &= s & (\text{Get-Put}) \\
v : V, s : S \vdash \text{get}(\text{put}(v, s)) &= v & (\text{Put-Get}) \\
v_1, v_2 : V, s : S \vdash \text{put}(v_2, \text{put}(v_1, s)) &= \text{put}(v_2, s) & (\text{Put-Put})
\end{aligned}$$

In plain english the rules translate to the following.

- (1) *Get-Put*: If we read a value then write this value there should be no change.
- (2) *Put-Get*: If we write a value then read a value we should get the value we wrote.
- (3) *Put-Put*: If we write a value then write another value we should only get the last value we wrote.

The first two qualities describe lenses which are *well-behaved* with the inclusion of the third rule giving us a *lawful* lens.

With this we can now expand our description of the field type to include how this type can be used. Adding to our previous example we end up with the following constructor. For clarification ‘Callable[[A], B]’ is equivalent to $f : A \rightarrow B$.

paper/code/constructorex2.py

```

1 class FieldType[Label_t: str, Out_t, Rec_t]:
2     label: type[Literal[Label_t]]
3     out_t: type[Out_t]
4     rec_t: type[Rec_t]
5     get_t: type[Callable[[Rec_t], Out_t]]
6     set_t: type[Callable[[Rec_t, Out_t], Rec_t]]

```

Listing 11: Expanded field type constructor pseudocode

As this pattern is broadly based on well studied and somewhat intuitive principles we see it often form the foundation for different kinds of wrapper types or type specifications which aim to find some abstract way to characterize and constrain the properties and behaviors of types. We can see this in the original implementation of Jluna in 12.

jluna/.src/usertype.inl

```

1 template<typename T>
2 template<typename Field_t>
3 void Usertype<T>::add_property(
4     const std::string& name,
5     std::function<Field_t(T&)> box_get,
6     std::function<void(T&, Field_t)> unbox_set
7 );

```

Listing 12: Jluna add property function

In the case of Jluna the upper template parameter T is that of the abstract record type, in Jluna these are called `Usertype<T>`. As the name implies `Field_t` is the type of the field, in our case this corresponds to T_x . The function then takes 3 arguments, a name of the property i.e. the label, the getter `box_get` and the setter `unbox_set`. An important takeaway from this example is that as all information is statically known the compiler specializes all property specifications of T at compile-time.

Given that we now have a way of expressing record types, and an informal way of describing thier fields we can use the idea of lenses introduced to create a more precise descriptor of how we would express the type of a record type. The first thing we introduced when starting to talk about the formal semantics was the idea of the dependent product type, just as we applied this as a means of defining a record structure so too can we apply it to define the structure of a lens. We can deliniate the structure of our lens into 3 parts.

- (1) *Type*: The type on which the lens operates.
- (2) *Semantics*: The semantics of the lens, i.e. the getter and setter.
- (3) *Laws*: The laws which the lens must follow, i.e. the lens must be lawful.

To express this formally we can define the lens type as follows.

Definition 3.11 (Lens Structure). Given a dependent $T(x)$ indexed over a set of labels L we can define the structure of a lawful record lens for this type as follows

$$\text{Lens}_L = \begin{cases} T : (l : L \vdash T_l) : \text{type} \\ \times (\text{get}_{T_l} : T \rightarrow T_l) \\ \times (\text{set}_{T_l} : T \times T_l \rightarrow T) \\ \times (\text{rw} : (x : T_l, \tau : T) \rightarrow \text{Id}_{T_l}(\text{get}_{T_l}(\text{set}_{T_l}(x, \tau)), x)) \\ \times (\text{wr} : (\tau : T) \rightarrow \text{Id}_T(\text{set}_{T_l}(\text{get}_{T_l}(\tau), \tau), \tau)) \\ \times (\text{ww} : (x_1 : T_l, x_2 : T_l, \tau : T) \rightarrow \text{Id}_T(\dots))^3 \end{cases} \quad (14)$$

A new construct introduce here is the idea of the identity type ‘ Id_T ’, this expresses the idea of equality between two terms of type T . Hence the inclusion of the functions ‘wr’, ‘rw’ and ‘ww’ act as laws over the type T . In other words we are encoding the laws that we want our lens to follow into the structure of the type over the records.

With this we have a baseline for how we can talk about the structure and dynamics of our record type from the more abstract perspective that we work with in the case of binding generators. Particular relevance in our definition of the Lens is the base type, i.e. the abstract record type. The abstract record type represents basic the language agnostic model of objects with both languages need to comform to in order to be bridged. There are various different approaches to ensuring this comformance to an abstract model that typically depend on the languages being bridged. A very basic approach if you have compatible languages is to simply generate bindings in a comformant manner as a preprocessing step via header parsing, in this instance a binding generator would usually either ensure that the generation process is implicitly conformant and expose idiomatic types, or it would generate wrappers which have the aforementioned added flexibility at the cost of idiomatic usage and depending on the implementation performance. The approach taken by Jluna is to generate idiomatic bindings where the conformance rules for the types structure are defined implicitly through the application of the type specification in the runtime generation process. Julia allows for this through its implementation of metaprogramming enabling the ability to define or specialize a set of types at runtime given an abstract description of different properties of this type.

3.2.7 Application: Jluna’s Internals. In the most abstract basic sense we can think of boxing and unboxing to simply express the idea of a cross language assignment. The general goal with binding generators, at least those which focus on very idiomatic language usage, is to provide enough abstractions such that working with two languages feels as if you are working with one. More concretely we can say that the goal is to mitigate, as much as is possible within certain constraints, the manual specification of bindings and type marshalling code for handling incompatibilities. Hence if we go back to our basic idea what we would then strive for would be something in the following form.

```
-----
example/assignmentgoal.cpp
-----
1 MyType my_type_object = foreign_function();
```

Listing 13: Cross language assignment goal

If we then compare this to the best analogue in Jluna we have this.

```
-----
example/jlunaproxyassign.cpp
-----
1 RGBA cpp_rgba = Main.safe_eval("return RGBA()");
```

Listing 14: Jluna proxy assignment

Here ‘Main’ represents the module scope and ‘safe_eval’ is a wrapper around the Julia C-API function which uses the runtime to evaluate a snippet of code and return anything specified in a low level representation. As might already be obvious then from the code itself we clearly have some implicit transformation happening. It should also be clarified that naturally for this to work we need an existing mirrored type representations. As we established before Jluna works quite explicitly by delineating its framework into specification and logic, the specification being the type lens. Furthermore we know that Julia itself has a variant of lenses, or more simply speaking it has accessor methods for objects. The main question that arises here what is the implicit transformation happening? The most apparent answer is well clearly its unboxing, if we have an implicit transformation for some low level representation then the assignment must be unboxing. While this is accurate it doesn’t really address the relevant more nuanced point of what unboxing and for that matter boxing really is.

We can think of it on an intuitive level by going back to the analogy between types of types as recipes. In this context we can think of a recipe as having 3 important parts, the ingredients, the instructions and the tools. The analogy here being that ingredients represent the abstract properties of our constrained type, the tools how we handle these ingredients, which leaves us with the instructions which explain how we use the tools in conjunction with the ingredients to get our final product, namely the desired type. Since we have the tools and ingredients, we then know that the instructions then must represent the logic of how we use these tools to construct a type. Unboxing and boxing then reduce to the question of, for a language *A*, given the ingredients (our type representation) and the tools (data accessors) how do we use these to construct the type in language *B* and vice versa.

Some important takeaways from this are that a type representation; regardless of any explicit specification; will ultimately always be a function of the tools, ingredients and instructions, i.e. the complexity of the type we can construct in the corresponding language only aligns with the specification insofar as their fundamental overlap. Secondly, there is a dependency between tools, instructions and generality of constructions. The more a tool is catered towards a specific means of construction the less dependence there is on the instructions though this comes at a cost of losing generality as the tool is inherently less flexible in its application. Finally, the instructions themselves are a function of the tools and ingredients, in the context of binding generators this means that the logic of how we construct a type is a function of the data accessors and the type representation.

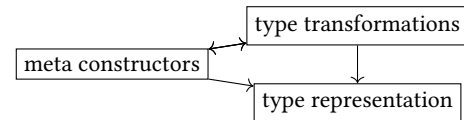


Figure 7: Basic dependencies of the binding generator

In 7, ‘meta constructors’ are just the type construction tools (e.g. lenses) and ‘type transformations’, in this context, is just boxing and unboxing. What follows from this is that, if we do want to extend a type representation; which in our case we do; we have to consider a few points.

- (1) *Ingredients:* We have to clearly define what the extended ingredients are for both languages, i.e. what are the new properties of the type representation.
- (2) *Tools:* We have to ensure that our tools are sufficient for working with the new ingredients, i.e. do we have the necessary data accessors.
- (3) *Instructions:* We have to define additional logic for how to use the tools in conjunction with the ingredients to construct the new type.

With this we have sufficient knowledge to understand how the original implementation follows from the ideas introduced. Broadly speaking we can consider two forms of types, fundamental and grouped. A record would represent a grouped type, having fields with either fundamental types or grouped types. As we can’t trivially bridge a record (as it has unbridged members) we must first bridge the fundamental types before the record. To bridge any type, there needs to exist a context in which we have access to our meta constructors, boxing and unboxing logic, and type representations. Furthermore we need some way to access this context. To approach taken by the author to approach this can be seen below

In listing 15 we can see how the author related the type specification to a context in which it could be applied to preform bidirectional mappings of this type. The dictionary ‘_mapping’ associates the field name ‘symbol’ (wrapped by a Symbol class) with two lambdas which capture the meta constructors (getters and setters) within their context through C++ lambda capture notation ‘[captured](...args) -> ret_ty { }’, and a wrapped type representation. In this case we can see that when ‘Field_t’ is a fundamental type then the box call will be to a simple boxing method,

```

jluna/usertype.inl

1  template<typename T>
2  template<typename Field_t>
3  void Usertype<T>::add_property(
4      ...
5  ) {
6      _mapping.insert({Symbol(name), {
7          [get](T& t_inst) -> unsafe::Value* {
8              return jluna::box<Field_t>(get(t_inst));
9          },
10         [set](T& t_inst, unsafe::Value* val) -> void {
11             set(t_inst, jluna::unbox<Field_t>(val));
12         },
13         Type((jl_datatype_t*) jl_eval_string(
14             as_julia_type<Field_t>::type_name.c_str())
15         )
16     });
17 }

```

Listing 15: Add property function

e.g. for primitive conversions. When ‘Field_t’ is a record type i.e. it has itself more fields (assuming it has a UserType instantiation) we call its box method, which in turn will call the box methods on its field. For completeness I have an adapted pseudocode version of the box and unbox approach of the original implementation in 16.

```

jluna/usertype.inl

1  template<typename A>
2  A Usertype<A>::bridge(B* in) {
3      auto out = A();
4
5      for (auto& pair : _mapping)
6          std::get<bridge_tool>(pair.tools)(out,
7              jluna::safe_call(
8                  B_get("accessor"), in,
9                  (B*) pair.symbol
10             )
11          );
12
13      return out;
14 }

```

Listing 16: Bridge implementation

The bridging code in 16 is the simplified representation of box and unbox whereby we simply iterate over the fields as specified in the mapping and for each field pass of our bridging logic to the meta constructors within the map who then either bridge the type directly by calling to the fundamental bridging code or as mentioned before call to the corresponding bridge method for the record type of a particular field and so on.

3.2.8 So whats the issue? A very core attribute of the implementation is that it fundamentally relies on nested records existing as fully walkable trees, in other words the bridging functions must always be supplied with concrete types such that there exists a path over an object which can be fully traced. Hence this implementation does

not account for the case where we encounter an abstract type. The problematic nature then being that our implementation of the BVH tree is fundamentally based on abstraction for its representation, or more concretely as stated before we have polymorphic nodes represented by an abstract class. The aim of the extension is then first and foremost to find a way of how we can tell this process what it should do if it does encounter an abstract type.

3.3 Part 3: We have a solution

Let’s start off our solution by creating a high level breakdown of the user-facing and internal flow of the library which can be seen in 8. We could tackle this problem in a linear fashion though this could end up being somewhat problematic since we could end up defining a specification which is misaligned with the actual type mirroring process. The issue here goes back to the different dependencies we have in the binding generator as shown in 7. Intuitively it makes sense to start with the abstract type representation, ensuring that it can even be mirrored in any functional sense, as both the meta constructors to create this type and the type transformations rely on there existing some fundamental analogue for type representation in both languages.

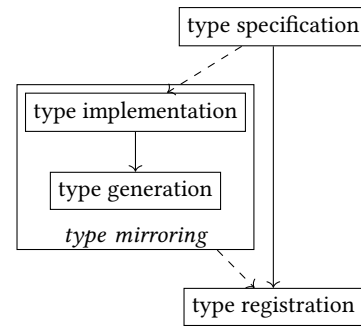


Figure 8: Stages of Jluna’s type binding generation process; --> represents user facing flow in how the user interacts with the library; → represents the internal flow of the library

3.3.1 Abstract Types and Substitutability. In our case there are two key inter-related things we wish to extend the library with. Firstly a support for abstract types, and from this a support for concrete types which implement this abstract representation. Abstract Data Type’s (ADTs) in the theoretical sense simply means to express a type which encodes a contract or model for a set of behaviors and properties which a concrete variant of this type must adhere to. A basic example could be the abstract definition of a list which is commonly parameterized with a type argument representing various concrete variants such a list of ints or a list of strings, the common semantic properties then being things like the ability to add, remove or access elements. A somewhat trivial benefit of this is that unless we must know the concrete type of a list we can work with this abstract definition and hence write more generic code which can work for a variety of concrete variants. Polymorphism is a natural byproduct that comes from the idea of substitutability of ADTs, if an ADT represents an abstract contract which a set of concrete types model then we can say that any type

following this contract could reasonably be substituted in place of this ADT without any change in behavior, this is the general idea of polymorphism or more specifically behavioral subtyping. A very strong condition of this idea was formalized by Liskov in her Substitution Principle.

Definition 3.12 (Liskov Substitution Principle). We denote $\phi(\tau_T)$ as a property provable about objects τ_T of type T . Then $\phi(\tau_S)$ should be true for objects τ_S of type S where S is a subtype of T denoted $S <: T$.

$$S <: T \rightarrow (\forall \tau_T : T) \vdash \phi(\tau_T) \rightarrow \phi(\tau_S)$$

As this is a quite strong condition its often relaxed in implementations of subtyping, it nonetheless serves as a good basic formalism for how to think about the idea of substitutability.

One common practical instance in which we see behavioral subtyping occur is in the case of interfaces. Interfaces within the context of programming languages refers to implementations of the aforementioned idea of an abstract type. Behavioral subtyping is then usually seen in the dynamics of interface implementation. For the case of many nominally typed object-oriented programming languages we see this take the shape of an abstract class representing the interface and an inherited concrete class forming the implementation. It should be noted that while inheritance can allow for subtyping it does not necessarily imply it, likewise the presence of subtyping does not imply inheritance. Another paradigm which has emerged often as a solution of the deficits that come with inheritance based subtyping is the idea of composition based subtyping, with a common phrase being ‘favor composition over inheritance’. Composition based subtyping is similarly based on the idea of an abstract interface which dictates a set of behaviors and properties which a concrete implementation must follow, the key distinction being that a concrete implementation does not inherit from the abstract interface but rather is composed of it. In other words, for composition based subtyping the concrete implementation exist as an instantiated version of the abstract interface directly rather than an inheriting subclass. Implementations of composition based behavioral subtyping are often seen in the form of protocols, traits or interfaces, all of which are used to express the idea of a contract which a concrete implementation must follow. To clarify again the difference here being that with composition based subtyping there is no hierarchical relationship between the abstract interface and the concrete implementation, the concrete implementation is simply a realization of the abstract interface. Composition based subtyping can be understood more broadly as an implementation of parametric structural subtyping or more simply generics in which substitutability is determined by the structure of the object which exists as a specialization of the general interface.

A very important point of consideration is that since subtyping is a function of a languages type system, how do its enforcement properties influence the dynamics of subtyping. We can categorize enforcement into two main qualities, when enforcement happens, and how it happens. Since we are concerning ourselves specifically with substitutability the question becomes when do languages enforce this and how do they enforce it. When it comes to the ‘when’ the trivial approach would be to delineate compile-time and run-time, though as JIT compilers in a sense combine the two it makes more sense to state it in terms of information availability. We can

define some hypothetical object o which is of type S , and we have a method taking an argument of type T where $S <: T$, clearly the most trivial variant of substitutability would occur in the case where $S = T$, and this is precisely what we tend to see with parametric structural subtyping, here methods are specialized such that for all concrete variants of a type T we have a corresponding concrete method accepting this variant. For statically typed languages we see this as generic methods being instantiated through means like the aforementioned monomorphism though it can also occur using runtime based approaches. In the case of dynamically typed languages there is a little more nuance depending on implementation, the untyped case, which is to say purely structural subtyping, is defined by the semantic properties of how the argument is applied. The typed case would refer to dynamic languages which have a gradual type system, here there is considerable variation in not only the execution method (interpreted, JIT compiled) but also the extent of enforcement and application. On one end of gradual typing systems we have TypeScript which exist in large part separate from the underlying execution of JavaScript and simply acts as a layer on top of JS. On the other end we have languages like Julia which; despite having a sort of gradual typing; makes considerable use of the type annotations as a means of more effectively specializing code with the aim of minimizing runtime compilations. Nominal subtyping on the other hand tends to be a predominantly run-time process regardless of language, the main reason being that with nominal subtyping we express substitutability as a function of an explicit hierarchy as opposed to direct static equality. The hierarchy is then something which must persist and be used for us to determine a valid substitution as opposed to it being trivially and apparent. A notable distinction being statically typed languages tend to enforce at compile-time that abstract methods be implemented in an inheriting class such that LSP doesn’t end up being violated, whereas dynamically typed languages might have their type systems enforce a nominal hierarchy via annotations but this does not necessarily imply that LSP is met for all properties. The basic takeaway from this is that we can in a sense interpret type systems as existing on two spectrums, the extent to which they are integrated and applied within the languages runtime, and the strictness of the invariants it enforces.

3.3.2 Substitutability in Julia and C++. What we see in Julia is that it enforces nominal subtyping of abstract types in only the most direct sense, which is to say for a type S and an abstract type T as long as there exists a declared subtyping relationship $S <: T$ then we can use S in place of T . The declaration has no enforcement on the actual underlying structure of the object, here Julia falls back to simple error checks if an property access does not resolve. More broadly we can say that within Julia subtyping enforcements are entirely a function of the types declaration as opposed to any structural qualities of the object. We can see this in more rigorous terms through the work of Zappa et. al. [27] which aimed to formalize Julias implementation of subtyping.

Definition 3.13 (Julia Subtyping). As defined by Zappa et. al. we can denote the type level representation of a user-defined type as follows

$$attr\ name\{t_1 <: T <: t'_1, \dots, t_m <: T <: t'_m\} <: t'' \in tds \quad (15)$$

Here *attr* expresses if a type is concrete or abstract, *name* is the name of the type. Following this we have the type parameters T each which have their defined lower bound t_i and upper bound t'_i , in addition to any type bounds applied to the type itself t'' . The type definition is then a part of global type declarations *tds*. The inference rule for application of subtyping is then defined as follows for a given typing context Γ

$$\frac{\text{APP-SUPER} \quad \begin{array}{l} \text{name}\{T_1, \dots, T_m\} <: t'' \in \text{tds} \\ \Gamma \vdash t''[a_1/T_1 \dots a_m/T_m] <: t' \vdash \Gamma' \end{array}}{\Gamma \vdash \text{name}\{a_1, \dots, a_m\} <: t' \vdash \Gamma'}$$

$$\frac{\text{APP-INV} \quad \begin{array}{l} n \leq m, \Gamma_0 = \text{add}(\text{Barrier}, \Gamma), \forall i \in [0, n] \\ \Gamma_{i-1} \vdash a_i <: a'_i \vdash \Gamma_i \wedge \Gamma_i \vdash a'_i <: a_i \vdash \Gamma_i \end{array}}{\Gamma \vdash \text{name}\{a_1, \dots, a_n\} <: \text{name}\{a'_1, \dots, a'_n\} \vdash \text{del}(\text{Barrier}, \Gamma_n)}$$

Here the rule APP-SUPER expresses the application of a subtype to a supertype, the rule APP-INV expresses the application of a subtype to an inverse subtype. APP-INV introduce some new notation, namely *add* and *del* of a ‘Barrier’; a ‘Barrier’ represents a tag which is used to delinate or distinguish between type variable introduced before and after the constructor. The check itself then simply ensures the constructor application is invariant.

The case of C++ is somewhat more complex, as its a statically typed language aside from enforcing a nominal hierarchy like Julia it also enforces a structural hierarchy so as to not violate LSP ahead of time. Informally we can say that given a type S and an abstract class of type T , for S to inherit from type T it must implement all the abstract methods of T , in a sense enforcing that its a valid substiution. A key difference from this to structural subtyping or some variants of parameterized structural subtyping is that while in the abstract context we loose any excess properties of the object. Stated differently inheritance based subtyping overlaps with parameterized structural subtyping if it enforces a concrete implementation of abstract methods, with the difference being we loose trivial access to excess properties (i.e. we need downcasting). To speak about this in more rigorous terms one approach we can take is first introducing the formalisms of structural subtyping and then seeing how inheritance based subtyping fits into this. If we consider a function in a dynamically typed language taking an object o and applying some set of behaviors which we can group as being described by some hypothetical type T . As structural subtyping only requires that an object passed as o only exhibits at least the properties of T we can imagine passing a bunch of objects which meet the requirements but also might have additional properties, if we construct a type T' which consists of the requires behaviors of T and the additional properties, then we can understand a structural subtype as being a type S which we can represent in T' as a composition of T and the additional properties of S . To express this formally we first introduce the idea of the type of all propositions and then build on this to define subtyping.

Definition 3.14 (Type of all propositions). The type of all propositions for a type S denoted Prop or Ω expresses a set of elements which are characterized by propositions which are true for S . In a simpler sense we can think of the type of all propositions as being

the set of all possible descriptions about a part of the whole of S which are true. For the most general definition we say that Ω is a type with a type family T such that for all propositions $P : \Omega$ the type $T(P)$ is inhabited, which is to say that it has a single member which acts as a proof of the proposition asserting that it is true.

$$\Omega : \prod_{P:\Omega} \prod_{a:T(P)} \prod_{b:T(P)} a =_{T(P)} b \quad (16)$$

The idea of subtyping then somewhat naturally follows from the idea of having a type subtype S of a type T if this type corresponds to some proposition in the set of all propositions of T . We can then define subtyping as follows.

Definition 3.15 (Subtyping). Given a type S and a type T having a type of all propositions Ω we can define a subtype of T as being a member p of the function type $P : S \rightarrow \Omega$ expressing a predicate which characterizes a subset of T i.e. the subtype. So we can say that

$$S \equiv \sum_{x:T} \text{El}(p(x)) \vdash S <: T \quad (17)$$

The notation $\text{El}(p(x))$ in this instance returns the element of the type family T which is inhabited by the proposition $p(x)$, in other words it returns x if $p(x)$ is true.

We can then elaborate and apply the general definition of subtyping to better understand the dynamics of structural subtyping. As a reminder we consider a valid structural subtype as a type which conforms to the required properties, and has additional properties. Hence we can think of an implicit structural supertype as being the type T who’s elements all have the required properties and then some arbitrary combination of excess properties. Keeping this in mind we can then define the structural subtype as follows.

Definition 3.16 (Structural Subtyping). A structural subtype S of a type T is a type with an injection / embedding $i : S \hookrightarrow T$, which is to say that S can be represent as a subset of T . The predicate which asserts this quality can be expressed as follows

$$x : T \vdash \sum_{y:S} i(y) =_T x \quad (18)$$

The analogy to structural subtyping in dynamically typed languages here being again that T represents our implicit structural supertype and substiutability is determined if this type exists within the supertype T . It should be clarified that this is not an expression of any type checking mechanism but rather an analogous formalism for how we can think about the dynamics of structural subtyping.

Having this defined lets finally approach how we can express inheritance based subtyping in the context of structural subtyping. As a reminder with inheritance based subtyping we loose any excess properties, expressing this in terms of structural subtyping the analogy would be that we collapse all valid structural subtypes into their intersection as this by definition resolves to the required properties. If we think of the abstract properties as an index type into a family of subtypes then we can express the intersection of these subtypes as follows.

Definition 3.17 (I-Indexed Intersection). Given a type T , an index type I , and a family of structural subtypes $i : I \vdash S_i <: T$ with a

family of embeddings $i_S : \prod_{i:I} S_i \hookrightarrow T$, we can define the intersection of these subtypes as being those members of S_i for which there exists a corresponding member in T .

$$\bigcap_{i:I} S(i) = \sum_{x:T} \prod_{i:I} \sum_{y:S(i)} i_S(i, y) =_T x \quad (19)$$

We can see that this intersection, is representative of a non-concrete subtype of T , with its concrete counterpart being represented by the depent product $\prod_{i:I} S(i)$. A valid downcast can then be understood as a function which takes the unified representation i.e. the intersection and returns the concrete representation. Presented informally as an inference rule we could express this as follows.

$$\frac{\text{DOWNCASE} \quad \Gamma \vdash i : I \vdash S_i <: T \vdash \bigcap_{i:I} S(i)}{\Gamma, \prod_{i:I} S(i) <: T}$$

Firstly it should be stated again that this is obviously a very abstract way of thinking about subtyping, but it offers some unique insights into how at least from a birdseye view certain attributes of type sytems only have very subtle differences in actuality but these differences can have a considerable impact on the actual practical day to day usage of a programming language. In this case the key takeaway is that inheritance based subtyping which enforces concrete implementations of abstract methods can be understood as a special case of structural subtyping in which we collapse all valid structural subtypes into their intersection which forms the abstract representation.

3.3.3 The extension - Fields. Given the above formalisms we can now approach how we can extend the library to support abstract types. We can work in a kind of top down fashion starting from the fundamental type representation to the actual bridging code. To bring us back to where we started we had the most basic form of our record represented as a type family indexed by a set of labels. We had 3 main components to this, the the label, the object type, and the dependent field type. The corresponding representation in C++ can then be seen in the template parameter list for the P template metafunction in listing 17.

```
src/cpp/usertype.cpp

1 template <typename Ot, typename Ft, const char* name>
2 struct P {
3     using obj_t = const Ot;
4     using field_t = Ft;
5     static constexpr const char* get_name() {
6         return name;
7     }
8     static std::function<Ft(Ot&> get;
9     static std::function<void(Ot&, Ft)> set;
10 };
```

Listing 17: Type meta-function

As a reminder, contrary to a typical function, a template meta-function is one which takes in a set of statically bound types as its arguments and returns a type as its result. The parameter list in this case is analogous to the base type component of the lens we developed in 14. The `obj_t` and `field_t` types are just trivial

access mechanisms which reflect the parameterized type back. The function `get_name` is required to access the field name from a static context. Finally we have the getter and setter methods, similarly to the name reflection these are declared as static methods. The only distinction in our case from the traditional implementation of a lens is that we do not return the updated object when setting a field as its just not necessary for our purposes.

To contrast this with the original implementation here instead of binding the field properties to the context of the `add_property` function we isolate them as an entirely static metafunction. There are a few benefits to this, but the core motivation being to simply create a further separation between how we specify a type and the tools to construct said type, and the actual logic applying these tools. Another benefit of this is that, while currently there exists no native reflection within C++ it should be possible to replace the manual specification of these metafunctions with automated ones with the full addition of reflection.

3.3.4 The extension - Types. A full type, or more concretely in our case object is then a composition of these fields. As metafunctions usually act on the type level as opposed to the value level we pass them to functions as template arguments. Here we have variadic templates come into play which allow us to pass an arbitrary number of our metafunctions to a function, in turn allowing us to construct this composition of fields. The original implementation divided this process of defining types into two parts, the C++ sided definition i.e. the type registration through the `add_property` function, and the Julia sided definition i.e. the type implementation and mirroring. As I wanted to generalize the field or property definition as a function which registers a type over a group of fields as opposed to for each field individually I adapted the logic into the `initialize_type` function, the signature of which can be seen in listing 18.

```
libs/jluna/include/usertype.hpp

1 template<
2     typename... P, template <typename...> class A,
3     typename... D_t, template <typename...> class B
4 >
5 static void initialize_type(A<P...>, B<D_t...>);
6
7 template <typename... D_t>
8 static void initialize_map();
```

Listing 18: Type initialization signature

One caveat we encounter when working with variadic functions is how to deliniate between distinct sets of arguments. In our case we needed to not only pass the set of fields as P but we also needed to pass the derived types relevant for a particular object to preform the necessary downcasts to loosing excess properties known. To deliniate between these two sets of arguments we define a variadic container as being a metafunction which simply acts a means of enclosing a set of types. In our case we can see these lists as the parameters `<typename...> class Name`, here we pass these lists as value level metafunctions which then allows us to define two distinct sets of type lists P and D_t with D_t representing our derived

types. At compile-time this merely acts as a way of specializing our type registration for the given specification, at runtime this specialization is then used to actually register all of the types for the C++ sided code.

The Julia sided type mirroring happens on the C++ through the implementation declaration. We can see the signature of this in the listing 19. The most relevant parts here are the two type parameters T referring to the type of the object and U being the supertype. On a high level we can think of this expressing to Julia that within a context of a module (the default just being the global scope) we want to establish the relationship $T <: U$. The type generation triggered through this call is then executed at runtime and calls into the Julia sided portion of the library to actually create a type with this relationship.

```
-----
libs/jluna/include/usertype.hpp
-----
1 template<typename T>
2 template<typename U>
3 void Usertype<T>::implement(unsafe::Module* module)
```

Listing 19: Type implementation signature

How we then use both this C++ sided initialization and its corresponding runtime registration and the Julia sided mirroring is shown in an example with listing 20. Here `tl` represents our basic type list containing the properties of the sphere type, the second argument is the relevant derived types for the properties, in this instance just the lambertian material. The implementation then expresses the relationship `sphere <: Hittable` and at runtime uses the information from the registration to mirror the type with the given properties.

```
-----
src/cpp/usertype.cpp
-----
1 Usertype<sphere>::initialize_type(
2     tl<
3         sphere_center,
4         sphere_radius,
5         sphere_r_squared,
6         sphere_mat,
7         sphere_bbox
8     >(),
9     tl<lambertian>())
10 );
11
12 Usertype<sphere>::implement<Hittable>();
```

Listing 20: User type example application

3.3.5 The extension - Bridging. The final and most major part of the implementation was extending the actual bridging process, both by generalizing the logic to work over objects as opposed to fields and by accounting for how to handle abstract types. Before delving into any of the specifics we have to first introduce a final mechanism in C++ which was heavily used within this process, this being the *fold expression*. As the name implies this mechanism enables the folding of a set of expressions into a single expression through some binary operation. In C++ this is divided into two forms of

folding, binary and unary with there being a further subdivision into a left fold and a right fold. The relevance of this mechanism to our extension is that since we are generalizing our process to work over a template pack as opposed to just single type arguments we need a way to work with this pack. In our case we use a unary fold with a ‘`,`’ operator. Per the specification an instantiation of a left unary fold expression is expressed as follows

$$(\dots op E) \rightarrow (((E_1 op E_2) op \dots) op E_n) \quad (20)$$

The comma operator can be understood in the context of a fold expression over a template pack as simply a way of chaining together or sequencing an expression over a set of types. So if we think of our expression E as being parameterized by a set of types T_i then the fold expression would simply be a way of applying E to each T_i in the pack. Expressed using the notation in 20 we could present this as follows

$$(\dots, E\langle T_i \rangle) \rightarrow (((E\langle T_1 \rangle, E\langle T_2 \rangle, \dots), E\langle T_n \rangle)) \quad (21)$$

In the context of instantiating a template pack this can also be viewed as a form of explicit or user defined monomorphism, which is to say as opposed to the compiler implicitly instantiating concrete variants of generics depending on the necessary ones we force an instantiation over a pack. Of course this isn’t exactly a 1:1 comparison but at least on a high level the underlying mechanisms are somewhat similar.

```
-----
src/cpp/usertype.cpp
-----
1 template<typename B, typename... D>
2 bool assign_shared_ptr(std::shared_ptr<B>& val, B* in) {
3     bool assigned = false;
4     (... , [&]() -> void {
5         if (auto d = dynamic_cast<D*>(in)) {
6             val = std::shared_ptr<B>{
7                 std::make_shared<D>(*d)
8             };
9             assigned = true;
10         }
11     }());
12     return assigned;
13 }
```

Listing 21: Variadic downcast and assignment

3.4 Part 4: How well does it work?

3.5 Limitations and Future Practical Work

4 DISCUSSION

5 CONCLUSION

REFERENCES

- [1] S. Akbary, M. McCaskey, J. Kruth, *et al.*, *Wasienv/wasienv*, Jun. 6, 2023. [Online]. Available: <https://github.com/wasienv/wasienv>.
- [2] S. Audet and Contributors, *Javacpp - the missing bridge between java and native c++*, GitHub repository, 2024. [Online]. Available: <https://github.com/bytedeco/javacpp>.

- [3] Ayke, R. Evans, D. Gryski, *et al.*, *Tinygo-org/tinygo*, Nov. 14, 2024. [Online]. Available: <https://github.com/tinygo-org/tinygo>.
- [4] Bytecode Alliance, *Cargo-wasi: A lightweight cargo subcommand for the wasm32-wasi target*, Provides tools for building, running, testing, and benchmarking Rust code for the wasm32-wasi target. Licensed under Apache-2.0 with LLVM exception., 2024. [Online]. Available: <https://github.com/bytecodealliance/cargo-wasi>.
- [5] Bytecode Alliance, *The webassembly component model*, <https://component-model.bytecodealliance.org/>, Accessed: 2024-11-15, 2024. [Online]. Available: <https://component-model.bytecodealliance.org/>.
- [6] Bytecode Alliance, *Wasmtime documentation*, Wasmtime is a standalone runtime for WebAssembly, WASI, and the Component Model. It is designed to be fast, secure, configurable, and standards compliant, and can run WebAssembly outside of the web environment., 2024. [Online]. Available: <https://github.com/bytecodealliance/wasmtime>.
- [7] R. Cooper, “Records and record types in semantic theory”, en, *Journal of Logic and Computation*, vol. 15, no. 2, pp. 99–112, Apr. 1, 2005, issn: 0955-792X. doi: 10.1093/logcom/exi004. [Online]. Available: <http://dx.doi.org/10.1093/logcom/exi004>.
- [8] C. Cords, *Jluna: A modern julia wrapper for c++*, <https://www.clemens-cords.com/jluna>, v1.0.0, 2022. [Online]. Available: <https://www.github.com/clemapfel/jluna>.
- [9] W. S. Fulton, O. Betts, VZ, *et al.*, *swig/swig*. Feb. 5, 2025. [Online]. Available: <https://github.com/swig/swig>.
- [10] Y. Futamura, “Partial computation of programs”, in *RIMS Symposia on Software Science and Engineering*. Springer Berlin Heidelberg, 1983, pp. 1–35, isbn: 9783540394426. doi: 10.1007/3-540-11980-9_13.
- [11] D. Gohman, A. Crichton, A. Brown, *et al.*, *Webassembly/wasi-sdk*, Oct. 29, 2024. [Online]. Available: <https://github.com/WebAssembly/wasi-sdk>.
- [12] Google, *Crubit: C++/rust bidirectional interop tool*, GitHub repository, 2024. [Online]. Available: <https://github.com/google/crubit>.
- [13] Google and Contributors, *Cxx - safe interoperability between rust and c++*, Project documentation, 2024. [Online]. Available: <https://cxx.rs/>.
- [14] F. Henglein, “Efficient type inference for higher-order binding-time analysis”, in *Functional Programming Languages and Computer Architecture*. Springer Berlin Heidelberg, 1991, pp. 448–472, isbn: 9783540475996. doi: 10.1007/3540543961_22.
- [15] B. Janssens, P. Jain, M. Horn, *et al.*, *JuliaInterop/CxxWrap.jl*. Jan. 29, 2025. [Online]. Available: <https://github.com/JuliaInterop/CxxWrap.jl>.
- [16] B. Kundu, V. Vassilev, and W. Lavrijsen, “Efficient and accurate automatic python bindings with cppy; cling”, 2023. doi: 10.48550/ARXIV.2304.02712. [Online]. Available: <https://arxiv.org/abs/2304.02712>.
- [17] nLab authors, *Product type*, <https://ncatlab.org/nlab/show/product+type>, Revision 24, Jan. 2025.
- [18] S. Pestov, *Compiling Swift Generics*. Aug. 2024, Explains the generics implementation in the Swift compiler.
- [19] S. H. Peter Shirley Trevor David Black, *Ray tracing in one weekend*, <https://raytracing.github.io/books/RayTracingInOneWeekend.html>, Aug. 2024. [Online]. Available: <https://raytracing.github.io/books/RayTracingInOneWeekend.html>.
- [20] S. H. Peter Shirley Trevor David Black, *Ray tracing: The next week*, <https://raytracing.github.io/books/RayTracingTheNextWeek.html>, Aug. 2024. [Online]. Available: <https://raytracing.github.io/books/RayTracingTheNextWeek.html>.
- [21] C. Rowley, github-actions[bot], M. Cranmer, *et al.*, *JuliaPy/Python-Call.jl*. Jan. 22, 2025. [Online]. Available: <https://github.com/JuliaPy/PythonCall.jl>.
- [22] SingleStore Labs, *Wasix: Posix compatibility layer for wasi builds*, Provides stubs for POSIX compatibility in WASI builds, facilitating the compilation of sources expecting POSIX support. Designed for experimental use., 2024. [Online]. Available: <https://github.com/singlestore-labs/wasix>.
- [23] swift-ci, D. Gregor, S. Pestov, *et al.*, *swiftlang/swift*. Feb. 8, 2025. [Online]. Available: <https://github.com/swiftlang/swift>.
- [24] A. Turcotte, E. Arteca, and G. Richards, *Reasoning about foreign function interfaces without modelling the foreign language (artifact)*, en, 2019. doi: 10.4230/DARTS.5.2.9. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/DARTS.5.2.9>.
- [25] E. Unruh, *Prime number computation*, ANSI X3J16-94-0075 / ISO WG21-462, 1994. [Online]. Available: <https://stackoverflow.com/questions/48849447/does-example-of-erwin-unruh-work-today>.
- [26] T. L. Veldhuizen, *C++ templates as partial evaluation*, 1998. doi: 10.48550/ARXIV.CS/9810010.
- [27] F. Zappa Nardelli, J. Belyakova, A. Pelenitsyn, B. Chung, J. Bezanson, and J. Vitek, “Julia subtyping: A rational reconstruction”, en, *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–27, Oct. 24, 2018, issn: 2475-1421. doi: 10.1145/3276483. [Online]. Available: <http://dx.doi.org/10.1145/3276483>.
- [28] E. Zhdan, *Swift/c++ interoperability*, <https://llvm.org/devmtg/2024-04/>, 2024 European LLVM Developers’ Meeting, Slides available at <https://llvm.org/devmtg/2024-04/slide...>, 2024. [Online]. Available: https://www.youtube.com/watch?v=2gfQd_OYUU&t=716s.
- [29] S. Zheng, Z. Zhou, X. Chen, *et al.*, “Luisarender: A high-performance rendering framework with layered and unified interfaces on stream architectures”, *ACM Trans. Graph.*, vol. 41, no. 6, Nov. 2022, issn: 0730-0301. doi: 10.1145/3550454.3555463. [Online]. Available: <https://doi.org/10.1145/3550454.3555463>.